universität koblenz
weiter:denken

Retrieval-Augmented Generation with Text–Symbolic Formula Indexing for Scientific Text

Masterarbeit

zur Erlangung des Grades eines Master of Science
im Studiengang Web and Data Science

vorgelegt von

Abram Jopaul

Erstgutachter:	Prof. Dr. Ralf Lämmel Institut für Informatik
Zweitgutachter:	Susanne Göbel Institut für Informatik

Koblenz, im September 2026

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit eigenständig und ohne fremde Hilfe angefertigt habe. Textpassagen, die wörtlich oder dem Sinn nach auf Publikationen oder Vorträgen anderer Autoren beruhen, sind als solche kenntlich gemacht.
Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

	Ja	Nein
Mit der Einstellung der Arbeit in die Bibliothek bin ich einverstanden.	☑	☐
Der Veröffentlichung dieser Arbeit im Internet stimme ich zu.	☑	☐



Koblenz, 30.09.2026

....

(Ort, Datum) (Abram Jopaul)

Zusammenfassung

Eine mathematische Frage wird in Worten und Formeln zugleich gestellt, und ein Retrievalsystem muss beides lesen. In einem gewöhnlichen Textindex steht eine Formel nur als die Zeichenfolge, mit der sie geschrieben wurde; die Anordnung, in der ihre Bedeutung liegt, fehlt dort. Die Struktur zu indexieren hilft bei der Formelsuche; ob es auch bei der Antwortsuche hilft, ist eine andere Frage, denn relevant ist eine Antwort, wenn sie die Frage löst, nicht, wenn ihre Formeln übereinstimmen.

Diese Arbeit baut drei strukturbewusste Formelindizes, einen je Formelrepräsentation, neben einem dünn besetzten und einem dichten Textindex und führt ihre Ranglisten in zehn Anordnungen zusammen. Die fünfzehn Konfigurationen werden auf der Antwortsuche über die 78 Topics der ARQMath-3-Kollektion bewertet, unter einem zuvor an den menschlichen Urteilen validierten Relevanzbewerter, und jede liefert anschließend einen Kontext an einen Generator unter einem festen Prompt.

Formelevidenz verbessert die Antwortsuche nicht. Kommt einer der drei Formelindizes zum stärksten reinen Textretrieval hinzu, bleibt das System dahinter zurück, ebenso bei allen dreien mit je eigener Stimme. Wie sie zusammengeführt werden, entscheidet über den Abstand: Bündelt man die drei Formelranglisten zu einer Stimme, zieht das System mit dem reinen Textretrieval gleich, und nichts hier übertrifft es. Diese Anordnung verdient ihren Platz, indem sie die Formelevidenz daran hindert zu schaden, nicht indem sie liefert, was den Textranglisten fehlt. Am Index liegt es nicht; bei der Formelsuche schneidet er vergleichbar mit einem veröffentlichten System ab. Die Formeln eines Topics stecken auf 70 der 78 Topics stärker in den relevanten als in den nicht relevanten Antworten, unter den am besten passenden aber, wo sich eine Rangliste entscheidet, nur noch auf 30. Formelübereinstimmung findet die richtige Nachbarschaft und kann innerhalb dieser nicht ordnen.

Ein abgerufener Kontext erreicht die erzeugte Antwort dennoch, die faktisch korrekter ausfällt als eine ohne Kontext geschriebene. Wie viel einer Antwort aus ihrem Kontext stammt und wie viel davon richtig ist, steigt mit der Qualität des Retrievals dahinter, wenn auch nicht gleichmäßig und nicht bei jeder Größe des Generators. Das Retrieval erreicht die Antwort; es schreibt aber nur wenig davon.

Abstract

A mathematical question is asked in words and formulas together, and a retriever has to read both. A formula enters an ordinary text index as the characters it was typed with. Rename the variables and the index sees an unrelated string. What the index never holds is the arrangement itself, and that is where a formula's meaning lies. Indexing that structure is what formula search does; whether it also helps when answers are what is retrieved is a separate question, because an answer is relevant when it resolves the question and not when its formulas match.

This thesis builds three structure-aware formula indexes, one per formula representation, beside a sparse and a dense text index, and merges the rankings they return in ten arrangements. Fifteen configurations in all are scored on answer retrieval over the 78 topics of the ARQMath-3 collection, under a relevance judge validated beforehand against the human assessors, and each of them then supplies a context to one generator under one fixed prompt.

Formula evidence does not improve answer retrieval. Adding any one of the three indexes to the strongest text-only retrieval leaves the system behind that retrieval alone, and so does adding all three when each of them votes separately. How the rankings are merged decides how far behind: collapsing the three formula rankings into a single vote before fusing draws level with text-only retrieval, and nothing built here passes it. That arrangement earns its place by keeping formula evidence from doing harm rather than by supplying what the text rankings lack. The failure is not the index, which performs comparably with a published system on formula search. A topic's formulas are contained in the answers judged relevant more than in those judged not relevant on 70 of the 78 topics, but among the best-matching answers, where a ranking is decided, on only 30. Formula matching finds the right neighbourhood and cannot order inside it.

A retrieved context nonetheless reaches the generated answer, which is more factually correct than one written from no context at all. Both how much of an answer comes from its context and how much of it is correct rise with the quality of the retrieval behind it, though not equally and not at every size of generator. Retrieval reaches the answer; it does not write much of it.

Acknowledgements

I wish to convey my deepest gratitude to Prof. Dr. Ralf Lämmel for his guidance and mentorship as my first supervisor. I extend my heartfelt thanks to my second supervisor, Susanne Göbel, for guiding and supporting me from the initial phase of the thesis through to its completion, and for her prompt suggestions on both my research and my writing. Without their help, I would not have been able to structure my thesis in this direction and complete it on time.

I would also like to extend my thanks to my friends and family for their support and encouragement during the challenging moments of my academic pursuit.

Contents

1 Introduction **1**

1.1 Motivation 1
1.2 Objectives 3
1.3 Research questions 4
1.4 Hypotheses 5
1.5 Contributions 7
1.6 Structure of the thesis 7

2 Related Work **9**

2.1 Mathematical information retrieval 9
2.2 Formula representation and retrieval systems 10
2.3 Retrieval-augmented generation 11
2.4 Evaluation with incomplete relevance judgments 12
2.5 Research gap 14

3 Theoretical Background **15**

3.1 The ARQMath collection and tasks 15
3.2 Formula representations 17
3.3 Indexing and retrieval 20
3.4 Rank fusion 21
3.5 Retrieval evaluation metrics 23
3.6 Relevance assessment and agreement 25
3.7 Generation evaluation metrics 28

4 Experimental Setup **31**

4.1 Topics, qrels and preprocessing 31
4.2 System overview 33

4.3 The scoring pipeline and its calibration . . . 34
4.4 The LLM relevance judge, and its validation . . . 36

5 Retrieval 44
5.1 Overview . . . 44
5.2 Method . . . 45
5.2.1 Formula representation and indexing . . . 45
5.2.2 Retrieval configurations . . . 48
5.2.3 Evaluation protocol . . . 50
5.3 Guards . . . 53
5.4 Results . . . 57
5.4.1 Overview . . . 57
5.4.2 Single-index retrieval . . . 57
5.4.3 Adding one formula index to text retrieval . . . 58
5.4.4 Adding all three formula indexes . . . 60
5.4.5 How the formula rankings enter the fusion . . . 60
5.4.6 Removing an index . . . 62
5.5 Where the formula signal lives . . . 64
5.6 What this settles about RQ1 . . . 66

6 Generation 69
6.1 Overview . . . 69
6.2 Method . . . 70
6.3 Results . . . 74
6.3.1 Overview . . . 74
6.3.2 What the retrieved context changes . . . 74
6.3.3 Retrieval quality and answer quality . . . 76
6.3.4 The size of the generator . . . 78
6.4 What this settles about RQ2 and RQ3 . . . 80

7 Concluding remarks 82
7.1 Summary . . . 82
7.2 Limitations . . . 84
7.3 Future work . . . 85

8 Acknowledgement of AI Tool Usage 87

List of Figures

4.1 System overview 35
4.2 Calibration of the relevance judge 42

5.1 Adding one formula index to dense retrieval 59
5.2 Adding all three formula indexes 61
5.3 Blocked against flat fusion 63
5.4 Containment of a topic's formulas 66

6.1 The answer-generation prompt 71
6.2 Answer quality against retrieval quality 77

List of Tables

4.1 The ARQMath-3 relevance judgments 32
4.2 Calibration of the scoring code 36
4.3 Validation of the relevance judge, by edition 39
4.4 The assessors' grades against the judge's 40

5.1 Tuples extracted from a worked formula 46
5.2 fastText settings for the tuple embeddings 47
5.3 The ten fused configurations 51
5.4 Formula search against a published system 54
5.5 Unjudged answers under the ARQMath labels 56
5.6 Each retrieval index on its own 58
5.7 The three formula rankings fused 62
5.8 Removing one index from each full system 64

6.1 Generation and scoring settings 72
6.2 What the retrieved context changes 76
6.3 The three generator sizes 78

Chapter 1

Introduction

1.1 Motivation

A student with a mathematical question asks it in words and formulas together. A language model can answer such a question from what it learnt in training, and it will answer whether or not it knows. A standard approach is to put a search step in front of it, so that the model is provided with the relevant context. The model then writes its answer from the question and that context [25]. Text is the most widely used retrieval source in systems built this way [18].

Mathematics is not prose. A post on a mathematics site carries its argument in two registers at once, the words that explain and the formulas that do the work, and a retriever has to read both. A formula enters an ordinary text index as the characters it was typed with, so it can be matched only where those same characters appear. Rename the variables and the index sees an unrelated string. The index never holds the arrangement itself — operators and their operands, and the spatial arrangement of variables and operators. This is where the meaning of a formula lies. This is the problem mathematical information retrieval has worked on since it began [11].

The solution is to index the structure rather than the characters. A formula can be parsed into a tree, and that tree turned into something an index can hold and compare, so that two formulas count as similar when they mean the same thing rather than when they are written the same way [12, 32]. An index of that kind is a structure-aware formula index. These indexes work when the query is itself a formula. Formula search — given one formula, return the formulas most

like it — is what they were designed for, and where they are compared against one another [31].

The task this thesis studies is the other one. Answer retrieval takes a mathematical question, stated in words and formulas, and returns the posts that answer it [50]. What separates it from formula search is not its input but its standard: an answer is relevant when it resolves the question, so a post whose formulas match the question symbol for symbol can still be worthless, and a restatement of the question is the clearest case of it. A formula index that finds similar formulas well has therefore not been shown to find answers well. The two cannot be read off one another.

The first difficulty follows from that, and it is one of arrangement. A formula can be represented as a tree in more than one way, and each of these formula representations keeps something the others drop, so a system can hold several of them at once, one index per representation, rather than choosing between them. Nor can a formula index work alone: the words around a formula are what separate an answer deriving a result from a comment repeating the question, so its ranking has to be combined with that of text-only retrieval — retrieval over the words of a post and nothing else. How several formula representations behave as a group, and how that group meets text retrieval, is underexplored where answers are what is retrieved, as Chapter 2 sets out.

The second difficulty is that a better ranking is not the end of the matter. What a reader sees is the answer the generator wrote, and a ranking reaches that answer only through the context it supplied. Those are two measurements, and whether the first carries into the second is unsettled: the published findings disagree [40, 19, 3], and none of that work varies how formulas are represented in the retriever.

The third difficulty belongs to the measuring rather than to the system. A test collection is judged once, on what the systems taking part at the time retrieved; everything outside that is left unjudged and normally counted as not relevant [6]. A system built afterwards retrieves posts nobody ever looked at, and is marked down for finding them. Nothing can be claimed about such a system until the instruments that score it have been shown to work, which is why two of them are established here before any result is reported.

The scope of what is claimed here should be clear from the start. The problem described above is general: wherever a document carries mathematics in its text, an index reading only the characters loses the structure, and the context a gen-

erator receives is the poorer for it. What this thesis measures is narrower. Every experiment runs on one collection of community mathematical question answering, built from a snapshot of Mathematics Stack Exchange [50]. The questions are asked by people who want help with a problem rather than written up for publication, the answers are posts rather than articles, and no scientific paper is retrieved anywhere in this thesis. Every claim made here is a claim about that collection.

What remains is a system to build and three things to find out about it. Section 1.2 states them.

1.2 Objectives

The objective is to build a retriever for answer retrieval that reads formula structure, and to find out what it changes — first in the ranking, and then in the answer a generator writes from it.

Three structure-aware formula indexes are built, one per formula representation, so that the representations are compared rather than assumed. They are placed beside two text indexes, one matching the words of a post literally and one matching them by meaning. A question is put to all of them, and the rankings they return are merged into one. A retriever that combines a text index with at least one formula index is a *multi-index retriever*. How the merge is done is not given in advance, so it is varied: which rankings enter it, and whether the three formula rankings enter it together or separately.

Each such retriever is scored against relevance judgments for the same questions, and that score is its retrieval quality. Whether adding formula indexes to text-only retrieval raises it is the first thing to settle.

The second is what that reaches. The best of the multi-index retrievers hands the top of its ranking to one generator as the retrieved context, and the answers are scored twice: for how much of what an answer says its context supplied, which is *faithfulness*, and for how much of what it says is correct, which is *factual correctness*. Both readings are set against answers the same generator writes for the same questions with no context at all — *generation without retrieval* — and against the answers built from text-only retrieval's context. The third thing to settle is whether any of it holds when the generator is larger.

Two things are outside the objective. The retriever is assembled from published components and none of them is trained further. One prompt is written and held fixed, so that what separates two sets of answers is the context and nothing else.

Section 1.3 puts all of this as questions.

1.3 Research questions

Three questions organise the work. The first asks whether structure-aware formula indexes improve answer retrieval over text alone, and whether the way they are combined governs the result. The second asks how much the retrieved context of the resulting retriever improves the answers a generator produces, and how faithful those answers are to it. The third checks that the second's answers do not depend on the size of the generator. Retrieval and generation are kept apart deliberately, each answered in a chapter of its own, so that retrieval is settled before generation is opened.

RQ1 Does adding structure-aware formula indexes to text-only retrieval improve answer retrieval on mathematical questions, and does the way those indexes are combined govern the effectiveness of the resulting multi-index retriever? Chapter 5

RQ1.1 Does adding one formula index to the strongest text-only retrieval improve answer retrieval?

RQ1.2 Does adding all three formula indexes to the strongest text-only retrieval improve answer retrieval, and improve on adding one, and does the way they are combined make a difference?

RQ2 To what extent does the retrieved context of the best multi-index retriever of RQ1 improve the factual correctness of generated answers over generation without retrieval and over the strongest text-only retrieval, how faithful are those answers to that context, and do both rise with the quality of the retrieval that supplied it? Chapter 6

RQ2.1 Does the context of the best multi-index retriever of RQ1 make answers more factually correct than generation without retrieval, and

more factually correct and more faithful than the strongest text-only retrieval?

RQ2.2 Across all the retrieval systems compared in RQ1, do factual correctness and faithfulness rise with retrieval quality?

RQ3 Do the answers to RQ2 hold as the generator grows from 1.5 to 7 billion parameters? Chapter 6

Two further conditions have to hold before any answer to these questions can be trusted, and neither is a research question in its own right. Both are established in the experimental setup.

P1 The evaluation pipeline reproduces the organisers' published figures on their own baseline run (Section 4.3).

P2 Retrieval built after the test collection was assembled can nonetheless be scored on a judgment pool it never contributed to — through hole-rate reporting, prime metrics and a relevance judge validated before it is relied upon (Section 4.4).

1.4 Hypotheses

The hypotheses divide as the questions do, into one group for retrieval and one for generation. Each is stated as a positive prediction that the data is free to reject. The retrieval hypotheses are ruled in Section 5.6 and the generation hypotheses in Section 6.4.

Retrieval

H1.1 (RQ1.1)

Each formula index, added to the strongest text-only retrieval, improves answer retrieval over that retrieval alone. Section 5.4.3

H1.2 (RQ1.2)

Adding all three formula indexes improves answer retrieval. **(a)** The three-index system improves on the strongest text-only retrieval. **(b)** It improves on adding one. Section 5.4.4

H1.3 (RQ1.2)

How the indexes are merged makes a difference: merging the three formula indexes into one vote before fusing, *blocked*, beats letting each of them vote separately, *flat*. Section 5.4.5

H1.4 (RQ1.2)

Every index contributes: removing any one of them makes the fused system worse. Section 5.4.6

Generation

H2.1 (RQ2.1)

With the prompt held fixed, the multi-index retriever's context makes answers more factually correct than generation without retrieval. Section 6.3.2

H2.2 (RQ2.1)

The multi-index retriever's context makes answers both more factually correct and more faithful than the strongest text-only retrieval's context. Section 6.3.2

H2.3 (RQ2.2)

The better the retrieved context, the more the answer relies on it: faithfulness rises with the quality of the retrieved context across configurations. Section 6.3.3

H2.4 (RQ2.2)

The better the retrieved context, the more correct the answer: factual correctness rises with the quality of the retrieved context across configurations. Section 6.3.3

H3.1 (RQ3)

The conclusions of RQ2 are not specific to the smallest generator. **(a)** Each comparison RQ2.1 makes keeps its direction at 3 billion and 7 billion parameters. **(b)** Factual correctness and faithfulness still rise with retrieval quality at both sizes. Section 6.3.4

1.5 Contributions

The main contributions of this thesis are as follows.

1. The construction of three structure-aware formula indexes beside a sparse and a dense text index, and a systematic comparison of the ways their rankings can be combined. Two things are varied one at a time: how the three formula rankings enter a combination, and which indexes are in it. Both decide what the combined system retrieves.

2. The validation of a language model as a relevance judge on the collection's four-level relevance scale. Whether its labels order systems as the assessors' labels do is tested separately from whether it agrees answer by answer, and the smallest difference between two scores it can resolve is measured and then applied wherever two systems are compared.

3. A diagnostic for whether a retrieval signal matches what a task counts as relevant. It contrasts how far the signal separates relevant from non-relevant answers over a topic's whole judged set with how far it separates them among the best-matching answers, where a ranking is decided.

4. An end-to-end measurement of what a difference in retrieval reaches in the generated answer. Every retrieval system compared here supplies a context to one generator under one prompt, beside answers written with no context at all, so that two things can be set against retrieval quality: how much of an answer is drawn from its context, and whether what it says is correct. The measurement is repeated at two larger generators, as a check that its answers are not specific to the smallest.

1.6 Structure of the thesis

Chapter 2 sets this work against research on mathematical information retrieval, formula retrieval systems, retrieval-augmented generation and evaluation with incomplete judgments, and closes on the gap these questions address.

Chapter 3 defines what the rest of the thesis uses: the collection and its two tasks, formula representations, indexing, retrieval and fusion, and the measures for a ranking and for a generated answer.

Chapter 4 fixes what the two experimental chapters share: the topics and judgments, the corpus as the system reads it, the system end to end, and the two preconditions.

Chapter 5 answers the first question: it builds the systems compared, answers two threats to their scores, reports the results, locates where the formula signal lies, and rules on each retrieval hypothesis.

Chapter 6 answers the second and the third, generating answers from those systems' contexts and scoring each for its correctness and for how much of it came from that context.

Chapter 7 summarises what the two answers say together, what they leave unsettled, and where the work goes next. Chapter 8 declares the use of AI tools.

Chapter 2

Related Work

2.1 Mathematical information retrieval

This thesis puts known components together in a new arrangement, so what it adds can only be seen against the problems the field already poses. This section describes those problems, the campaigns that defined them, and the difficulties they share.

Mathematical information retrieval is the problem of searching documents that contain mathematical notation. It grew out of digital mathematical libraries, where ordinary text retrieval treated a formula as a string of characters and could not reach the structure that carries its meaning [11]. A query may be text, a formula, or both, which makes it a different problem, not a harder one.

Three tasks recur. Formula search takes one formula as the query and returns formulas like it. Answer retrieval takes a mathematical question, stated in text and formulas, and returns passages that answer it. Document retrieval, the earliest campaigns' task, takes keywords and formulas together and returns paragraphs of scientific articles [49]. Recent benchmarks add premise selection and statement retrieval over formal mathematics, and find every model weaker where reasoning is needed [22]. A recent survey groups them by one distinction: whether a system finds mathematical content or answers a question with it [48].

Shared campaigns defined them. The NTCIR Math tasks came first, from 2013: a pilot over a hundred thousand arXiv papers, then two further editions that added English Wikipedia [2]. The ARQMath labs, at CLEF from 2020 to 2022, moved the corpus to Mathematics Stack Exchange and set answer retrieval

beside formula search [50, 30, 31]. Participants there routinely merged several rankings into one, and the strongest automatic answer-retrieval run of the final edition was such an ensemble [31].

Three difficulties run through all of it. Notation is ambiguous: a small set of symbols stands for an immense number of concepts, so one expression can mean different things in different documents. The same idea can be written in several ways, so two formulas that mean the same thing may look nothing alike. And relevance is costly to establish: the NTCIR organisers drew their assessors from third-year and graduate students of pure mathematics, and divided documents into paragraphs to hold down the cost of judging them. Their account of the campaign opens by naming the absence of large-scale datasets, carefully designed tasks and established evaluation methods as the obstacle it set out to remove [2].

Those difficulties fall unevenly across the tasks. A formula query asks for formulas like the one given, so structural similarity is most of the problem; a question carries text and several formulas, and a useful answer need not resemble any of them. The organisers' baselines show it. Tangent-S, run in both tasks, reached 0.540 at formula search and 0.159 at answer retrieval, against 0.272 for a plain text baseline [31]. The two tasks are scored on different topics, so the pair shows the same system is much weaker at answer retrieval. The representations may not carry the similarity answer retrieval rewards; or they may, and no one has combined them with text retrieval well enough to reach an answer. No team submitted an answer-retrieval system built on mathematics alone, so the leaderboards cannot tell them apart. Section 2.2 turns to the representations.

2.2 Formula representation and retrieval systems

Section 2.1 left the representations themselves as the open question. Tangent-S is where the line starts: it indexes two tree representations of a formula separately and scores a candidate on both, and Davila and Zanibbi report that using the two together retrieves better than either alone on the NTCIR-12 collection [12]. That result, on formula search in 2017, is the published precedent for treating several representations as complementary. Tangent-CFT replaced the matching with learned vectors and merged its representations by summing them into one [32]. TangentCFT2 reversed that choice, holding four representations in four indexes

and merging their rankings instead, inside formula search, where the formula system is the whole system [28].

MathBERT pre-trains a language model over formulas and the text around them [34]; a graph-contrastive method learns formula embeddings with no relevance labels at all [47]. Both report gains over Tangent-CFT, and both were measured on NTCIR-12, a different collection with different queries and judgments, so neither figure can be set beside the ARQMath ones. SSEmb is the exception, and the strongest result of the three: it combines a formula's structure with its surrounding text and reports 0.734 nDCG′@10 at ARQMath-3 formula search, the highest among the automatic systems on its own recomputation of the leaderboard [26]. That figure is not on the lab's measure, so it cannot be set beside the published nDCG′ either.

What these representations have not been asked is how they behave as a group. TangentCFT2 is the one system that holds several formula representations apart, and it does so only in formula search, where there is no text to combine them with. Its own team submitted five answer-retrieval runs at the same lab, and none of them used a formula index at all: they searched the raw text of the posts, with formulas left in it as written [28]. So the grouping of formula representations, and the way that group meets text retrieval, has not been varied and measured where answers are what is retrieved.

This thesis rebuilds the Tangent-CFT-style stack from scratch and keeps its representations as separate indexes. Its formula embeddings are trained on the corpus's own formulas, and nothing in it is trained against relevance labels. What it varies is how those representations are grouped, so the learned systems above are not what it sets out to beat. Every system in this section is measured on the ranking it produces. None asks whether a difference visible at the ranking survives into the answer a language model writes from what was retrieved. Section 2.3 takes that question up.

2.3 Retrieval-augmented generation

A retriever used to be the last stage of a search system. It is now often the first stage of a system that writes the answer itself, the arrangement Lewis et al. set out and named retrieval-augmented generation [25]. So what gets judged is the

answer and not the ranking that produced it, and those are two different measurements.

Whether the first carries into the second is unsettled. Salemi and Zamani scored retrievers with the usual ranking measures against human relevance annotations, compared those scores against how well the surrounding system answered, and found rank correlations no higher than about 0.2 across five widely used benchmark collections, often close to zero [40]. That result is hard to check against most published work, because most of it reports only the end of the pipeline: the generated answer is compared against a known correct answer, and how well the retrieval step did on its own is never stated, so an improvement cannot be traced to one part or the other [40]. Frameworks such as RAGAS are a recent attempt to make the generation side measurable at all [13].

On mathematics the picture is different again, and thinner than it looks. Henkel and colleagues built a tutoring system for middle-school students over a free pre-algebra textbook, annotated how relevant each retrieved section was, and over fifty-one questions found no relationship between that relevance and which answers readers preferred [19]. A recent Olympiad-level benchmark reports the opposite for problem solving, where answer quality is highly sensitive to how good the retrieval is [3]. Both search curated corpora of prepared material by semantic similarity. Neither varies how formulas are represented in the retriever, so neither says whether a formula index reaches the answer.

ARQMath ran a generation task of its own in 2022, over the same questions as its answer retrieval task. A baseline that retrieved nothing and answered from the model alone scored higher than every participating system [31]. The organisers also note a difficulty with reusing what they collected: the judgments made there cannot be applied to later systems in the way retrieval judgments can, because the measures depend on matching a new answer against one already judged [31]. That difficulty is not particular to generation, and Section 2.4 takes it up.

2.4 Evaluation with incomplete relevance judgments

No collection of any size is judged completely. There are far too many documents, so the systems taking part submit their rankings and the documents near the top

go to the assessors. Everything else is left unjudged, and unjudged normally counts as not relevant. That works for the systems that were there at the time. A system built afterwards retrieves documents nobody ever looked at, and is marked down for finding things that may have been good answers.

One response is to change the measure rather than the labels. Buckley and Voorhees proposed bpref, which scores a ranking on its judged documents alone [6]; Sakai and Kando later deleted judgments from real collections to see which measures still ordered systems correctly, and found the condensed family most robust [39]. Both set aside what nobody looked at, and both are safe only if the unjudged documents are no better and no worse than the judged. Buckley and Voorhees decline to conclude that bpref fairly evaluates a system which did not take part, since a new system can break that assumption [6].

The other response is to supply the missing labels. Thomas and colleagues showed that a carefully written prompt produces labels close to those of trained assessors on web search, and how delicate that is: across thirty-two variants, agreement ranged from κ 0.20 to 0.62 [44]. An open-source reproduction, UMBRELA, was then run across a public evaluation of seventy-seven runs from nineteen teams beside the organisers' own assessors [45]. Rankings of whole systems came out close; agreement topic by topic was weaker, the model was more lenient, and having humans check its labels did not improve the match [45].

That result is disputed in three ways. Clarke and Dietz set that overall rank correlation of 0.89 against the 0.56 they find among the top fifteen systems — where the comparison matters [8]. Fröbe and colleagues compared eight automatic assessors. They agreed with each other more closely than any of them agreed with a human, and they favoured systems that used a language model to rank, placing those on average nine to seventeen positions too high [17]. And Abbasiantaeb and colleagues labelled only the documents nobody had judged, then used those labels beside the assessors' own; that mixture tracked the assessors worse than labelling every document automatically did [1].

All of that evidence comes from web search, general question answering and conversational search. Where judging needs specialist knowledge the picture is worse: in two professional fields, subject-matter experts agreed with an automatic judge on roughly two thirds of cases, 68% and 64% [43]. Anyone relying on automatic labels therefore has to show they work on the collection in hand rather than point at results from another.

2.5 Research gap

The first gap is on the retrieval side. Formula representations are built and tuned for formula search, and that is where they are compared against one another. Systems built to retrieve answers have combined formulas with text before. In each of them the formulas were handled by a single index, not several. The only system that keeps several formula representations apart does so in formula search, where there is no text to combine them with [28]. So the grouping of those representations, and the way that group meets text retrieval, is untried where answers are what is retrieved.

The second gap is at the far end of the pipeline. Whether better retrieval yields better answers is unsettled in general, and on mathematics it has been measured only over curated corpora of prepared material, textbook sections and problem sets retrieved by semantic similarity [19, 3]. None of that work varies how formulas are represented in the retriever. So whether a difference that is visible at the ranking survives into the answer, on mathematics, has not been asked.

The third gap is in whether any of it can be measured. A system built after a collection was judged is scored against judgments made for other systems, and the two remedies, changing the measure or supplying the missing labels, both rest on assumptions the literature has not settled. Automatic labels have been tested on web search, on general question answering and on conversational search. Where judging needs specialist knowledge, they agree with human experts less often than those experts agree with one another [43]. No one has tested them on mathematical answers, where relevance turns on whether an explanation actually helps.

Chapter 3

Theoretical Background

3.1 The ARQMath collection and tasks

Every experiment in this thesis is run on the ARQMath test collection, so its properties determine what can and cannot be measured here. This section describes the corpus, the two retrieval tasks it defines, and the relevance criteria behind each.

ARQMath is a community question answering (CQA) collection built from a March 2020 snapshot of Mathematics Stack Exchange, covering questions and answers posted between 2010 and 2018 [50]. It contains roughly one million questions and 28 million formula instances, each formula carrying a unique identifier and distributed alongside pre-computed LaTeX, Presentation MathML and Content MathML representations. All three lab editions — ARQMath-1 (2020), ARQMath-2 (2021) and ARQMath-3 (2022) — search the same collection of posts, and differ in their topics, their relevance judgments, and the corrections the organisers made to the formula representations [31].

Answer retrieval (ARQMath Task 1) asks a system to rank up to 1,000 answer posts for a mathematical question. Each edition draws its topics from questions posted to Mathematics Stack Exchange after the corpus snapshot, so their own answers are not in the collection — ARQMath-3's come from 2021 — and only answer posts from 2010–2018 are searchable. Candidate topics had to contain at least one formula and have at least one moderator-annotated duplicate question inside the collection, so that assessment effort was not spent on topics with no relevant answer at all [31]. For ARQMath-3, 100 topics were released, 80 were

assessed, and two were removed because only a single answer had been judged relevant, leaving 78 topics in the evaluated test collection [31].

Formula search (ARQMath Task 2) takes a single formula from one of those questions as the query and returns formula instances drawn from both questions and answers. Both post types are searched because a retrieved formula is only ever a route to text: one occurring in an answer may sit in a worked solution, and one occurring in another question routes the searcher to that question's own answers, so restricting the search to answers would discard half the routes to useful material. The ARQMath-3 test collection has 76 assessed topics [31]. Relevance is contextual: a candidate formula is judged inside the post it appears in, and the query formula inside the question it was taken from. Evaluation is performed over *visually distinct* formulae rather than instances — those that render identically are clustered under a shared visual identifier and credited once — so retrieving twenty copies of the same expression earns no additional reward [29]. A third pilot task, open-domain question answering, was introduced in 2022 and is not used here.

Both tasks use the same four relevance levels but define them differently, and the difference is the point. In answer retrieval, an answer is High if it is sufficient to answer the complete question on its own, Medium if it provides some path towards a solution, Low if it only helps find or interpret an answer, and otherwise Not relevant — a level the criteria explicitly extend to any post that restates the question without adding information [31]. In formula search, a formula is High if it is as good as an exact match to the query would have been, Medium if useful but less so, and Low if there is some chance of finding something useful through it [31]. Answer-retrieval relevance is thus the extent to which an answer resolves the question, and formula search asks how likely a formula is to lead a searcher to text that helps answer the question its query was taken from. Neither is structural similarity: a formula matching the query symbol-for-symbol can be judged non-relevant if its post is unhelpful, and a structurally dissimilar formula can be highly relevant. They are not equally far from it, though. The formula-search scale uses an exact match as its reference point: a formula earns the top grade when finding it would have been as useful as finding the query formula itself. A system that ranks by structural closeness is ranking candidates by how near they come to that exact match, so the system and the scale are pointed at the same thing. The answer-retrieval scale never mentions formulas at all.

Two binarisations of these levels recur throughout. *H+M* counts only High and Medium as relevant, matching the usual TREC notion, while *H+M+L* additionally counts Low; binary measures in this thesis use H+M, that is, relevance level 2 (Section 3.5).

Judgments were produced by pooling over the participants' *runs*. A run is the ranked list of answers a participating system submits for every topic; each team designates one of its runs as primary and the others as alternate. For each topic the organisers took the top 45 results of every primary run and the top 20 of every alternate run, merged and deduplicated them, and had trained assessors judge the result, yielding pools averaging 464 answers per topic for answer retrieval in ARQMath-3 [31]. The judgments are distributed as *qrels* (query relevance judgments), one entry per judged topic–document pair carrying its grade, so a qrels file records the pool as well as the grades: a document is judged exactly when it has an entry. The pool is therefore defined by what the 2022 participants retrieved, and any document outside it is unjudged rather than non-relevant. Such documents are called *holes*, and the fraction of a ranking's top-ranked results that consists of them is its *hole rate*, made precise at a cutoff in Section 3.5; incomplete judgments of this kind are a long-standing problem in pooled evaluation [6]. The organisers' response was to adopt measures that ignore holes rather than scoring them as irrelevant: the primary measure for both tasks is nDCG', the prime variant of Sakai and Kando [39], with MAP' and $\mathrm{P}'@10$ as secondary measures. That convention suffices for the systems that were pooled, but not for a system built afterwards, which retrieves documents no 2022 participant retrieved and therefore carries a far higher hole rate; Sections 3.5 and 4.4 take up that gap.

3.2 Formula representations

A formula written in LaTeX is a string, and string matching cannot see that $x - y^2$ and $a - b^2$ are the same expression under different names. Retrieving on structure therefore requires parsing the formula into a tree first. LaTeX is presentation source: it fixes which symbols appear and where they sit on the page, but encodes no structure a parser can read off directly. *Mathematical markup* does encode it, and the standard for it is MathML, an XML language for mathematics that comes in two flavours. Those two flavours support two different trees, and the difference between them runs through the rest of this thesis: Presentation MathML

records how a formula is laid out on the page, while Content MathML records what it computes. ARQMath distributes both for every formula in the collection (Section 3.1), so the trees can be read from the collection rather than parsed out of LaTeX.

The *Symbol Layout Tree* (SLT) is derived from Presentation MathML and encodes spatial arrangement. Its nodes are the symbols of the formula and its edges are the spatial relations between them — one symbol follows another on the same baseline, or sits above it as a superscript, or below it, or within a structure such as a radical.

The *Operator Tree* (OPT) is derived from Content MathML and encodes operator–operand structure instead. Its root is the top-level operator or relation, its internal nodes are the remaining operators, and its leaves are operands; each edge records which argument of its parent the child is. The OPT recognises y^2 as an exponentiation however it was typeset, but no longer knows what the expression looked like.

Both trees are linearised the same way. Following Tangent-CFT [32], which extends the symbol pairs of Tangent-S [12] with the path from the root, a depth-first traversal converts a tree into a list of *path tuples*, each pairing a node with one of its descendants, and every tuple, in either representation, has the same four positions:

`[0]` a node;

`[1]` a node that descends from it;

`[2]` the edge labels on the path from the first to the second;

`[3]` the path from the root of the tree to the first node.

What fills those positions is what separates the two representations. In an SLT, positions `[0]` and `[1]` hold a symbol written with its type — `V!x` for the variable x and `N!2` for the number 2, with `!` separating type from value — while operators such as the minus sign appear as the bare symbol `-`. Position `[2]` names the spatial relation: `n` for the next symbol on the same baseline, `a` for above, `b` for below, `w` for within, among others. Position `[3]` is then the sequence of those relations taken from the root, so `nn` is two steps along a baseline, and a tuple whose first node *is* the root carries `-`. In $x - y^2 = 0$, for instance, the tuple `(V!y,`

`N!2, a, nn)` records that the 2 sits above the y, and that the y lies two steps along the baseline from the root.

An OPT fills the same four positions differently. Its operators are marked for whether the order of their arguments matters: the subtraction in $x - y^2 = 0$ becomes `O!minus`, because $x - y^2$ and $y^2 - x$ are different, and the equality becomes `U!eq`, because $x - y^2 = 0$ and $0 = x - y^2$ are not. Position `[2]` gives the argument index, `0` for the first argument and `1` for the second, and position `[3]` the sequence of those indices from the root, so `01` is the first argument of the root followed by the second of its child. In the same formula, `(O!SUP, N!2, 1, 01)` records that the 2 is the second argument of the exponentiation, which itself sits at `01`: it is the second argument of the subtraction, and the subtraction is the first argument of the equality.

Both close their paths with a dummy marker, written `0!` — at the end of each baseline in an SLT, and at each leaf in an OPT — so that short expressions still produce matchable tuples.

How far from a node the traversal looks is set by a *window*. With a window of one, a node is paired only with its children, so the SLT of $x - y$ yields, besides its end marker, `(V!x, -, n, -)` and `(-, V!y, n, n)`. With a window of two, each node is also paired with the descendants two edges away, and $x - y$ yields `(V!x, V!y, nn, -)` as well, with position `[2]` recording both steps. A wider window captures relations between symbols further apart, at the cost of more tuples per formula.

The third representation, *SLT-TYPE*, is the SLT with its values erased and only the node types kept: every variable collapses to `V!` and every number to `N!`, so that `(V!x, -, n, -)` becomes `(V!, -, n, -)`. Operators carry no type marker in an SLT and so survive intact: what is forgotten is which symbols the operators act on, not the operators themselves. This makes $x - y^2$ match $a - b^2$ exactly, which is the intent, and also makes $x + x$ indistinguishable from $x + y$, which is the cost.

The three representations therefore sit on an abstraction ladder. SLT is the most literal and matches on appearance; OPT ignores notation but keeps operator identity; SLT-TYPE keeps the operators and forgets the symbols they act on. Each representation finds more formulas that are written differently but are related to some extent. Which step is the better trade-off cannot be settled by argument; it has to be measured. Tangent-CFT sidesteps the choice altogether by summing

the vectors of all three representations into one [32]. This thesis does the opposite: it builds a separate index for each representation and compares them against one another, so that the contribution of each can be seen on its own (RQ1.3). Section 3.3 describes how a tuple sequence becomes a vector that can be indexed and searched.

3.3 Indexing and retrieval

Retrieval methods differ in how a document is represented and how its similarity to a query is computed, and each representation needs a different kind of index to be searched at scale. Three families bear on this thesis: *sparse retrieval* over the terms a document contains, *dense retrieval* over learned vectors, and retrieval over the structure of the formulas a document holds. All three do the same thing — score every document in the collection against a query and return a ranked list — and this section describes how each represents a document, how each is searched, and what each can and cannot match. The third is the only one of the three that reads a formula as anything but a string.

Sparse retrieval scores documents by the query terms they contain, and BM25 is the standard weighting of this kind. BM25 weights a term by how often it occurs in the document, how rare it is across the collection, and how long the document is; two parameters shape the weighting, k_1 for how quickly repeated occurrences of a term stop adding score, and b for how strongly a document is penalised for its length [37]. Matching is lexical, so a document is scored only if it literally shares a term with the query. For mathematical text this cuts both ways: a formula matches only when its LaTeX tokens appear verbatim, but when they do the match is exact and cheap.

Dense retrieval replaces terms with meaning. A neural encoder maps the query and each document to a single fixed-length vector, and similarity is the dot product between them. The architecture is a *bi-encoder*: query and document are encoded independently, which is what makes the approach practical, since every document can be encoded once offline and only the query has to be encoded at search time [23]. Encoders of this kind are pre-trained on general text and then fine-tuned on large collections of question–answer pairs, which is what adapts them to the genre of a community question answering collection. Their strength

is that they match paraphrases a lexical match misses; their weakness is that a whole answer post, however long, must be compressed into one vector.

Formula retrieval embeds the path tuples of Section 3.2. The tuple sequence of a formula is treated as a sentence and each tuple as a word, an approach introduced by Tangent-CFT [32], and a *fastText* model is trained over the resulting corpus — a word-embedding method that learns a vector for each token from the tokens surrounding it, and that composes each token's vector from its character n-grams as well as the token itself, rather than from the token alone [4]. The training objective is skip-gram: each tuple is trained to predict the tuples around it and to distinguish them from randomly sampled ones, so tuples occurring in similar surroundings end up with similar vectors. Because a token is assembled from its n-grams, a tuple never seen in training still receives a vector. That matters here because mathematical expressions are close to unique, and a query formula absent from the collection is the normal case. A formula is represented by the mean of its tuple vectors, and two formulas are compared by the cosine of theirs.

Dense and formula retrieval need a different index from a sparse one. An inverted index scores only the documents sharing a term with the query, whereas a vector representation gives every document a non-zero similarity, making retrieval a nearest-neighbour search over the whole collection. Exhaustive search is exact but costs time proportional to the number of vectors, which is affordable for the answer posts and not for the roughly 28 million formula instances. *Approximate nearest-neighbour* search buys that cost back: the vectors are grouped into cells around learned centroids so that only the cells nearest the query are scanned, and each vector is stored compressed so that the index fits in memory [21]. Both steps trade a little recall for a large saving in time and space, and how much is traded is a parameter rather than a property of the method; Section 5.2.1 gives the settings used. Each of the three then returns a ranked list, but on scales that cannot be compared with one another, which is the problem Section 3.4 addresses.

3.4 Rank fusion

Mapping those scores onto a common scale is the obvious repair, and it is not free. A BM25 score is unbounded and depends on collection statistics, a dot product depends on the norms of two embeddings, and a cosine is confined to $[-1, 1]$. Adding or averaging such scores is meaningless until they are made commensu-

rable, and every way of making them so is itself a modelling decision: min–max normalisation, for instance, depends on the highest and lowest score that happened to appear in that particular result list, so the same document can be worth a different amount depending on what else was retrieved alongside it. Rank-based fusion avoids the choice altogether by discarding the scores and keeping only the position each document occupies, which is comparable across rankings by construction.

Reciprocal Rank Fusion (RRF) is the simplest and most widely used method of this kind. Given a set of rankings R, each document d receives the score

$$\mathrm{RRF}(d) = \sum_{r \in R_d} \frac{1}{k + r(d)}, \tag{3.1}$$

where $R_d \subseteq R$ is the set of rankings in which d appears, $r(d)$ is the rank of d in ranking r, and k is a constant, set to 60 in the original work [10]. The reciprocal shape means a document gains most from appearing near the top of a list and little from appearing deep in one, while k damps the difference between the highest positions so that no single ranking can decide the outcome by placing one document first. A document retrieved by several rankings accumulates contributions and rises; a document found by only one of them still enters the fused list. A ranking whose values are on a wildly different scale cannot dominate the result, because each contribution is bounded by $1/(k+1)$. That is precisely what makes RRF usable across sparse, dense and formula rankings without calibrating any of them against one another. The same peculiarity of RRF cuts the other way too. A contribution depends only on position, so the document a ranking places first is worth $1/(k+1)$ whether that ranking is a good one or a poor one: the rule weighs rankings equally because it has no measure of their quality, and a ranking that retrieves badly therefore pushes its own top documents into the fused list rather than merely failing to improve it.

Which rankings are worth combining, and with what value of k, the rule does not settle for itself; Section 5.2.2 states the choices this thesis makes. What no fusion rule settles at all is whether the list it produces is better than the ones that went into it — a question nothing inside a ranking can answer, and the one Section 3.5 takes up.

3.5 Retrieval evaluation metrics

A retrieval measure compares a ranked list of documents against the relevance judgments for the topic that produced it. ARQMath judges on the four-level scale of Section 3.1, so each judged document carries a grade $g \in \{0, 1, 2, 3\}$, and a measure turns a ranking and those grades into one number per topic, averaged over topics to score a system.

Two kinds of measure are standard on a graded collection of this sort. One counts how many relevant documents reach the head of a ranking; the other also reads the order in which they arrive.

Precision at k is the simpler of the two. It binarises the grades by the H+M rule — a document counts as relevant when $g \geq 2$ — and reports the fraction of the top k documents that are relevant:

$$\mathrm{P@}k = \frac{1}{k} \left| \{ d \in \text{top-}k : g(d) \geq 2 \} \right|. \tag{3.2}$$

It ignores both the grades above the threshold and the order within the top k: a ranking with three relevant documents scores the same whether they occupy the first three positions or the last three.

Normalised discounted cumulative gain keeps both [20]. Each document contributes a gain equal to its grade, discounted by how far down the ranking it sits, and these are accumulated:

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{g(d_i)}{\log_2(i+1)}, \tag{3.3}$$

where d_i is the document at rank i. The discount written here is the one that applies from rank 1 [7], and is what `trec_eval` computes; the original formulation leaves the first ranks undiscounted [20]. The logarithmic discount means a High answer at rank 1 is worth more than the same answer at rank 10, which is the property precision lacks. DCG is unbounded and depends on how many relevant documents a topic happens to have, so it is normalised by the largest value any ranking of that topic's judged documents could achieve — the *ideal* DCG, obtained by sorting all judged documents for the topic by grade and taking the first k:

$$\mathrm{nDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}. \tag{3.4}$$

A score of 1 means the top k could not have been ordered better, and 0 that nothing relevant was retrieved. The ideal is a property of the judgments, not of the ranking being scored: a relevant document added to a topic's judgments raises that topic's ideal for every system at once. Two nDCG values are therefore comparable only when they are computed against the same judgments.

Mean average precision reads the ranking to its end rather than stopping at a cutoff. For a single topic, average precision adds up the precision at every rank holding a relevant document and divides by the number of relevant documents the topic has:

$$\mathrm{AP} = \frac{1}{R} \sum_{i \,:\, g(d_i) \geq 2} \mathrm{P@}i, \tag{3.5}$$

where R is that number. MAP is the mean of AP over topics. It binarises at H+M as precision does, and rewards relevant documents that arrive early as nDCG does, but it is the only one of the three whose value depends on the whole ranking; it is also one of the measures ARQMath publishes for its participant runs [31].

All three measures assume that every document they read carries a grade.

Prime variants

A test collection judges a pool assembled from the systems that took part in the original evaluation. A system built afterwards retrieves documents that were never in that pool, and those documents have no judgment at all. Treating them as non-relevant penalises exactly the systems that retrieve differently from the pool, which is the case relevant to this thesis (Section 5.3).

The *prime* variants answer this by condensing the list: every unjudged document is removed from the ranking, and the measure is computed over what remains, so that the cutoff k is reached using judged documents only [39]. The ideal is untouched: it is still computed from every judged document for the topic, so condensing changes what a ranking is scored on, not what it is scored against. Written P′@k, nDCG′@k and MAP′, they are the official measures of the ARQMath labs [31] and are used throughout this thesis. Sakai found condensed-list measures more robust to incomplete judgments than the alternatives designed for the same problem [38], bpref among them [6].

Condensing is not free. Removing unjudged documents implicitly assumes they would have been graded like the judged ones around them, and where that

assumption fails the result is optimistic: condensed-list measures tend to overestimate systems that did not contribute to the pool [16]. The size of the assumption is quantified by the *hole rate*, the fraction of a ranking's top k that carries no judgment,

$$\mathrm{hole@}k = \frac{1}{k}\,\big|\{d \in \text{top-}k : d \text{ unjudged}\}\big|, \tag{3.6}$$

which says how much of the ranking a prime score was in fact computed over. Section 4.4 describes the second response to the same problem, which is to have the unjudged documents labelled rather than discarded. Since nDCG values are comparable only under shared judgments, that response must also settle which judgments every system is then scored against, and Section 5.2.3 settles it.

3.6 Relevance assessment and agreement

Section 3.5 ended on a problem. A system built after a collection was assessed retrieves answers no assessor saw, and discarding them overestimates it. The other response is to have those answers labelled by someone other than the original assessors, and increasingly that someone is a language model.

Whether such labels can stand in for the assessors' is a question that can be answered empirically, and it comes in two forms that are easy to confuse. The first is whether the two labellers give the same grade to the same answer. The second is whether scoring systems with one labeller's grades orders them the way the other's grades do. This section first introduces language models as relevance assessors, then gives a statistic for each of the two questions, and ends with how small a difference between two systems an evaluation can detect. The two questions have to be kept apart: a labeller can fail the first and still pass the second.

Relevance is usually judged by people, and human assessment is slow and expensive. That is why pools stop at a fixed depth (Section 3.1). A large language model can be prompted to do the same task: given a query and a document, assign one of the collection's relevance grades. A model used this way is an *LLM relevance judge*, or simply a *judge*.

Thomas et al. showed that a carefully prompted model can match human relevance preferences in web search [44]. UMBRELA, an open-source reproduction of their prompt, was used to assess the TREC 2024 RAG track alongside NIST's

assessors [45]. Its prompt describes the task, sets out each relevance level in turn, and forces a single grade as output. The comparison with NIST's assessors established three things: the judge reproduced the ordering of systems closely; it agreed with the assessors much less well on individual documents; and it was more lenient, rating documents relevant that the assessors did not. Its authors concluded that such a judge can replace manual judgments to measure effectiveness "in a coarse-grained manner", but should not be used to separate systems whose effectiveness is close. Each of these findings needs a statistic, and the rest of this section supplies them.

Agreement on individual answers

When two labellers grade the same items, their agreement is summarised in a *confusion matrix*. It has one row per grade the first labeller used and one column per grade the second used, and each cell counts the items that received that pair of grades. The diagonal holds the items on which the two agree. The share of all items that sit on the diagonal is the *observed agreement*, p_o.

Observed agreement flatters any labeller when one grade dominates. In a pooled collection most judged documents are not relevant. Two labellers who never read a document but answered "not relevant" every time would agree on most of them.

Chance agreement, p_e, measures that floor. Call the two labellers a and b, and write $P_a(k)$ and $P_b(k)$ for the share of items each of them assigns grade k. If the two assigned grades independently, at those rates, they would agree on grade k with probability $P_a(k)\,P_b(k)$, and on some grade with probability

$$p_e = \sum_k P_a(k)\,P_b(k). \tag{3.7}$$

Cohen's κ removes this floor [9]:

$$\kappa = \frac{p_o - p_e}{1 - p_e}. \tag{3.8}$$

The numerator is the agreement achieved beyond chance, and the denominator is the agreement that was available beyond chance. κ is 0 for a labeller no better than chance, 1 for perfect agreement, and negative when the two disagree more than

chance would. On a graded scale, κ can be computed over the grades as given, which asks whether the two chose the same grade. It can also be computed after binarising, for instance by the H+M rule of Section 3.1, which asks only whether they put the item on the same side of the relevance threshold.

κ is not a property of the labeller alone. Because p_e is built from how often each grade is used, the same observed agreement yields a smaller κ when one grade dominates. In that case chance alone already accounts for most of the agreement, so p_e is close to p_o, and κ comes out low even when the two labellers agree on most items. This is the *prevalence effect* [41], one of the two paradoxes of high agreement but low κ [15]. Two consequences follow. Comparing κ values from collections with different relevance rates is misleading. And κ can be interpreted only beside the p_o and p_e it was computed from.

Agreement on the ordering of systems

The second form of the question sets the labels aside and asks about their use. Each system in a set is scored twice with the same measure, once under each labeller's grades, and the two resulting orderings of the systems are compared.

Kendall's τ compares them pair by pair [24]. For every pair of systems, the two orderings either agree on which system is better, making the pair *concordant*, or disagree, making it *discordant*. With C concordant and D discordant pairs,

$$\tau = \frac{C - D}{C + D}. \tag{3.9}$$

No two systems tie on the measure used here, so $C + D$ is simply the number of pairs, and this coincides with Kendall's original coefficient. τ is 1 when the orderings are identical, 0 when they are unrelated, and -1 when one reverses the other.

Agreement on labels and agreement on orderings are different quantities, and either can be high while the other is low. Consider a labeller whose disagreements all run in one direction, for example one that grades every non-relevant document one step too high. It raises every system's score together, so it can leave the ordering nearly intact while its κ is low.

Resolving power

A high τ says that most pairs of systems keep their order. It does not say which pairs do not. Those are, predictably, the pairs whose scores are close. The ability of an evaluation to tell such close systems apart is its *resolving power*, and Upadhyay et al. found UMBRELA's to be limited [45].

Buckley and Voorhees give a way to measure it [5]. They compared the orderings that different evaluations of the same systems produce, in their case the same measure over different query formulations and over topic sets of varying size. For each pair of systems they counted how often the two evaluations disagreed about which was the better, and took the minority verdict as the error; the *error rate* is the total of those errors over all the decisions made, ties included. Close pairs are set aside by a *fuzziness value*: the percentage difference between two scores below which the scores are treated as equivalent. The closest pairs are the ones most likely to be misordered, so raising the fuzziness value lowers the error rate, at the price of leaving more pairs undecided. Together, a fuzziness value and the error rate it yields state how large a difference must be before the ordering it implies can be trusted.

Two questions therefore remain for the judge this thesis uses: whether a judge that disagrees with the assessors on individual labels can still be trusted to rank systems, and how small a difference it can resolve. Section 4.4 settles both.

3.7 Generation evaluation metrics

Retrieval measures say whether the right documents were found; they say nothing about what is then done with them. In *retrieval-augmented generation* a language model — the *generator* — receives the retrieved documents as its *context* alongside the question and writes the answer that is scored, so the answer needs measures of its own. Two questions arise once it exists, and neither answers the other: whether the answer is grounded in the context it was given, and whether the answer is correct. An answer can be grounded in a context that is itself wrong, and it can be correct from knowledge no context supplied. This section defines a measure for both.

Faithfulness is defined by RAGAS, a framework that scores the stages of a retrieval-augmented pipeline with a language model in place of a human anno-

tator [13]; factual correctness was added later to its implementation [14], in the version named in Section 6.2. The two are built alike, so the construction is given once and each measure follows from it. A text is first decomposed into *claims* — standalone assertions, each of which can be checked on its own. Every claim is then judged against some body of text, scoring 1 where it can be inferred from that text and 0 where it cannot. The language model performs both steps. What separates one measure from another is which text is decomposed and what its claims are checked against.

Faithfulness answers the first question: the generated answer is decomposed, and its claims are checked against the retrieved context, taken as one body of text. *Factual correctness* answers the second, and needs something to be correct about — a *reference*, a text holding the answer the question should have received. The answer is decomposed again, and its claims are checked against the reference instead. Its precision reading, *factual-correctness precision* (FC-P), is the share of those claims the reference supports. The two measures are therefore one formula read twice,

$$\begin{aligned} \text{Faithfulness} &= \frac{\big|\{c \in C(\text{answer}) : c \text{ inferable from the context}\}\big|}{\big|C(\text{answer})\big|}, \\ \text{FC-P} &= \frac{\big|\{c \in C(\text{answer}) : c \text{ inferable from the reference}\}\big|}{\big|C(\text{answer})\big|}, \end{aligned} \tag{3.10}$$

where $C(x)$ denotes the set of claims the text x decomposes into. Both denominators count the answer's own claims; only the text the claims are checked against changes.

What each measure leaves out follows from that. Faithfulness requires no correct answer to compare against. That is why it can be computed wherever a context exists, and why it is not a quality score. It measures grounding rather than truth: an answer scores 1 as long as the context supports every claim it makes, even when the context is wrong. Factual-correctness precision is silent in the other direction. It never sees the context, so an answer the generator wrote from what it already knew scores as well as one the context supplied, provided the reference supports it.

The denominator is where faithfulness is most easily misread, and the misreading is worth naming. The fraction is taken over the claims of the *answer*,

never over those of the context, so nothing about how much of the context was used enters the score: an answer that leans on one line of a long context and an answer that draws on all of it can both score 1. Faithfulness is a proportion of what was said, not of what was available.

One property of the shared construction bears on every score it produces. How finely a text is cut into claims is not fixed but governed by two parameters: *atomicity*, how far a sentence is split into separate assertions, and *coverage*, how much of a sentence's detail survives the split. Numerator and denominator are both counts of claims, so the granularity of the decomposition moves the whole fraction, and two answers of identical content can score differently if they are cut differently. That is a property of the instrument, not of the answers.

Faithfulness needs only a context. Factual correctness needs a reference, and for a mathematics question the collection does not hand one over ready-made: the same question usually admits several valid routes to the same result, so taking any one correct answer as the reference would mark a correct response wrong for having taken a different route. What a reference is built from instead is settled in Section 6.2.

Chapter 4

Experimental Setup

The retrieval and generation chapters each answer one research question, and each is meant to be read as a complete argument of its own. That works only if what the two share is fixed before either begins, and fixing it is what this chapter is for. Chapter 3 defined what the collection, the formula representations, the retrieval and fusion methods and the evaluation measures are; this chapter states what this thesis did with them that both questions depend on. Whatever only one question uses is described in that question's own chapter: how the indices are built and combined belongs to retrieval, and how answers are generated and scored belongs to generation.

Section 4.1 fixes the material — the topics, the relevance judgments taken from ARQMath, and the corpus as the system reads it — and Section 4.2 describes the system end to end, marking which part of it each question studies. Sections 4.3 and 4.4 establish the two preconditions of Section 1.3. The first is that the scoring code is correct. The second is that a language model can judge relevance well enough to rank systems the way the human assessors do. Every retrieval score in this thesis rests on both, and the generation chapter reads retrieval quality through those same scores, which is why neither precondition belongs to retrieval alone.

4.1 Topics, qrels and preprocessing

Both research questions are answered on the same answer-retrieval topics and scored against judgments from the same collection, so which topics and which

Topics	High	Medium	Low	Not relevant	Total
78	867	2,076	4,921	26,983	34,847

Table 4.1 The ARQMath-3 answer-retrieval relevance judgments: the number of judged (topic, answer post) pairs at each relevance level, over the 78 topics of the evaluated test collection.

judgments are used fixes what the numbers in the two chapters that follow mean. Section 3.1 described the collection, its tasks and how its judgments were pooled. This section states which of its topics and judgments this thesis uses, and for what, and how the corpus is turned into the files the system reads.

Every retrieval and generation experiment on answer retrieval in this thesis is evaluated on the 78 topics of the ARQMath-3 test collection (Section 3.1); Table 4.1 summarises the organisers' relevance judgments for them. Formula search keeps its own topics and judgments, and the retrieval chapter describes them where it uses them. One further set of judgments is used, and this thesis produces it rather than taking it from ARQMath: the relevance judge's labels for the answers the retrieval systems return. Section 5.2.3 describes how they are pooled, once the judge itself has been introduced.

The judgments of the two earlier editions, ARQMath-1 and ARQMath-2, enter the thesis in one place only: together with those of ARQMath-3, they are what the relevance judge is validated against. In all three editions at least three quarters of the judged pairs are Not relevant, and High is the rarest label; over the 78 topics, 26,983 of the 34,847 pairs are Not relevant and 867 are High. Relevance is rare, and Section 4.4 returns to what that rarity does to the judge's agreement with the assessors.

The collection is distributed as a single XML file, `Posts.V1.3.xml`, in which each post is one `row` element holding the post as HTML, every formula marked up in place and tagged with its collection identifier. Each post in the collection is marked as either a question or an answer (the `PostTypeId` attribute, 1 or 2), and the parse writes the two to separate JSON Lines files: 1,445,495 answers and 1,020,585 questions. Parsing strips the markup and leaves the LaTeX of each formula in place in the plain text, and additionally collects those formulas into a separate per-post list, so that one pass yields both the text that sparse and dense retrieval search and the formula list that formula retrieval needs. An answer

record keeps its identifier, the identifier of the question it answers, its community score, the body text and the formula list. The community score is retained because the reference used for generation evaluation ranks candidate answers by it.

The topics are parsed from the ARQMath-3 Task 1 topic file into a third JSON Lines file, each record keeping the topic's title and question body with their formulas handled as in the posts.

These are the inputs both research questions share: the answer posts the system searches, the topics it answers, and the judgments it is scored against. How the system turns a topic into a ranked list of answer posts, and how the top of that list is handed to the generator as context — where retrieval ends and generation begins — is what Section 4.2 lays out.

4.2 System overview

The retrieval and generation chapters study two parts of one system, and each reads as a complete argument only if it is clear where its part begins and ends. This section describes the system end to end, in Figure 4.1, and marks the point at which one research question hands over to the other. The hand-over fixes what each chapter varies and what it holds still.

The system works in two phases. The first happens once, offline, before any topic is seen, and builds the indices. Each of the answer posts of Section 4.1 is indexed three ways: its text in a sparse index scored by BM25, the same text as a single bi-encoder vector in a dense index, and its formulas in a formula index. There are three formula indices rather than one, one for each of the tree representations of Section 3.2, because each representation keeps different information about the same formula, and whether that difference matters for answer retrieval is a question the retrieval chapter takes up. The same formula embeddings are also indexed over every formula in the collection, to be queried with a single formula. That is formula search, and it lets the formula index be examined on the task its representations were designed for before it is asked to retrieve answers.

The second phase happens once per topic. The topic's text is sent to the sparse and dense indices and its formulas to the formula indices, so a single topic yields five ranked lists of answer posts. The reciprocal rank fusion of Section 3.4 merges two or more of them into one, or a single list is used on its own; either way the topic ends with one ranked list, and retrieval ends there. Generation starts from

the head of that list. Its top k posts become the context of Section 3.7, and the generator, a language model, answers the topic's question given that context and one fixed prompt.

The hand-over falls at the ranked list, and it divides the figure into its two bands. RQ1 studies everything above it — which indices are searched, which formula representation is used, how the lists are fused — and scores the ranked list itself. The generator plays no part in that: every score in the retrieval chapter is computed from what retrieval returns, before any answer has been generated, so nothing on the generation side can change it. RQ2 studies everything below the hand-over and varies two things. The first is the context: where its posts come from, or whether there is a context at all. The second is the size of the generator, since RQ2 also asks whether what retrieval adds depends on how much the generator already knows. The retrieval side is held fixed throughout. Each ranked list reaches the generator exactly as the retrieval chapter produced it, with no index rebuilt and no list re-ranked, and the prompt and decoding settings are the same for every answer. The generator changes only between comparisons, never within one, so that two answers to the same topic from the same generator differ only in what they were given to read.

The right-hand column of the figure shows what each question is scored with. A ranked list is scored with the prime measures of Section 3.5, against relevance labels from the assessors and from a relevance judge; an answer is scored with the RAGAS measures of Section 3.7, faithfulness against its context and factual correctness against a reference. The generation chapter uses the retrieval chapter's scores as well: to ask whether better retrieval produces better answers, it sets each ranked list's score beside the faithfulness and factual correctness of the answers generated from it. Both questions therefore rest on the same retrieval scores. That is why the parts shaded grey — the material of Section 4.1 and the two instruments behind those scores — are described in this chapter rather than in either of the next two. Whether the first of them, the scoring code, is correct is the question Section 4.3 settles.

4.3 The scoring pipeline and its calibration

Every retrieval score in this thesis comes from scoring code written for it, and both research questions read those scores: the retrieval chapter to compare retrieval

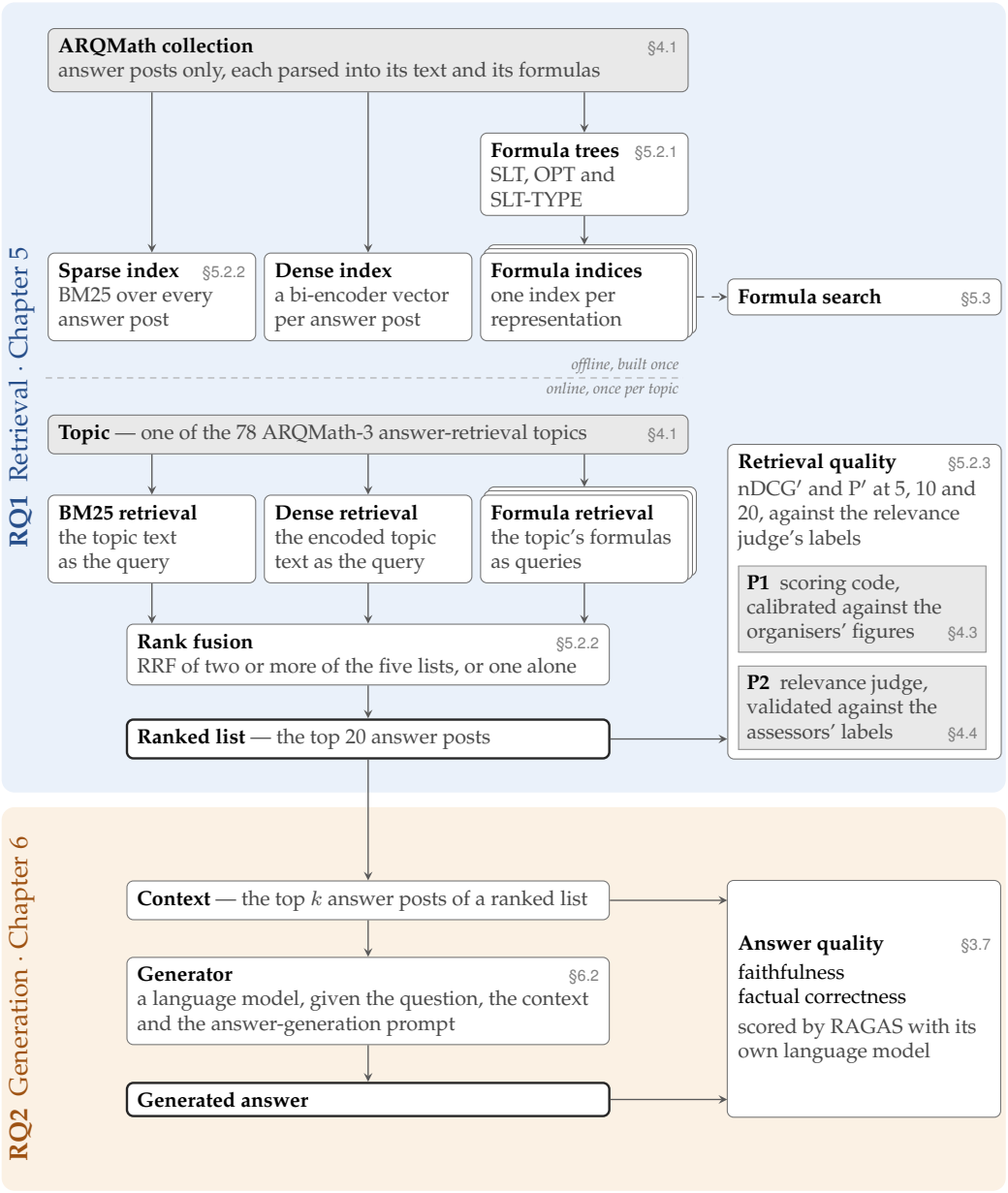


Figure 4.1 The system end to end. RQ1 (upper band) scores the ranked list; RQ2 (lower band) scores the answer generated from its top k posts. Grey boxes are shared by both. SLT, OPT, SLT-TYPE: symbol layout tree, operator tree, and SLT with its values erased; RRF: reciprocal rank fusion; nDCG′, P′: prime variants of normalised discounted cumulative gain and precision; P1, P2: the preconditions of Section 1.3.

Measure	Computed here	Published	Difference
nDCG′	0.2718	0.272	−0.0002
MAP′	0.0640	0.064	0.0000
P′@10	0.1244	0.124	+0.0004

Table 4.2 Calibration of this thesis's scoring code on the TF-IDF (Terrier) baseline of ARQMath-3 answer retrieval, scored against the organisers' judgments for the 78 topics of Section 4.1 and compared with their published figures [31, Table 3]. All measures are prime variants, computed with unjudged answers removed: nDCG′, normalised discounted cumulative gain, and MAP′, mean average precision, both over the full ranking; P′@10, precision in the top ten. Difference: computed minus published.

systems, the generation chapter to relate retrieval quality to answer quality. This section fixes how the measures of Section 3.5 are computed, and shows that the code computes them correctly.

The measures are computed with `pytrec_eval`, a Python interface to NIST's `trec_eval` [46]. All are taken in their prime variants: nDCG′ with the grade as gain, P′ and MAP′ at the H+M threshold of Section 3.1 (`relevance_level = 2`). Each is computed per topic and averaged over the topics with equal weight.

To show that this code is correct, it is run on a ranking whose scores are already known: the TF-IDF baseline, built with Terrier, that the ARQMath organisers distributed for ARQMath-3 Task 1 and scored in the lab overview [31]. Only the scoring code is this thesis's; the ranking, the judgments for the 78 topics of Section 4.1 and the figures to match are the organisers'. Table 4.2 gives the result: all three measures reproduce the published values to the precision at which those were printed, and the remaining differences are rounding.

This establishes precondition P1 of Section 1.3: given a ranking and a set of judgments, the code returns the number the organisers' own evaluation would. Whether those judgments can score a system built after they were made is the question Section 4.4 takes up.

4.4 The LLM relevance judge, and its validation

This thesis scores its own retrieval systems with relevance labels that a language model assigned rather than the ARQMath assessors, and this section is why that is allowed. Section 3.5 showed the problem it answers. The ARQMath judgments

cover the pool the 2022 participants retrieved. A system built afterwards retrieves answers outside that pool, and the prime measures, by discarding those answers, overestimate exactly such a system. The alternative Section 3.5 named was to have the unjudged answers labelled instead. That is admissible only if labels from a model rank systems the way the assessors' labels do, which is precondition P2 of Section 1.3.

This section describes the judge, tests it against the assessors' own judgments, and measures how close two systems can be before the judge can no longer order them. The retrieval and generation chapters apply that last measurement every time they compare two scores.

The judge

The judge is `gemini-3.5-flash-lite`, queried with one fixed prompt at temperature zero, so that the same question and answer always get the same label. The prompt follows the structure UMBRELA established for relevance assessment by language models [45]: a descriptive framing, the relevance levels set out one at a time, and a forced single-label output. It replaces UMBRELA's web-search notion of relevance with ARQMath's own.

The model is cast as an expert mathematician helping the student who asked. It is asked how useful the answer would be for resolving that student's question: not whether the answer is well written, and not whether it is the best possible answer. The four levels follow the ARQMath assessment guidelines, and four worked examples, one per level, fix the boundaries between them. Formulas reach the judge as unnormalised LaTeX. The model gives brief reasoning and then a final line naming the label, which is parsed from that line. An output with no parseable label leaves its answer unjudged.

Labels are requested in batches and cached under the topic, the answer, the prompt and the model, so each answer is judged only once for a given topic.

One boundary needs stating. The generation chapter also relies on a language model, inside the RAGAS measures of Section 3.7, to judge generated answers. That is a different instrument doing a different task, and nothing in this section validates it.

Validation against the assessors' judgments

The judge is validated where the right answer is already known: on the answers the ARQMath assessors judged in all three editions of the lab. Of 118,055 judged (topic, answer) pairs, 240 produced no parseable label. The remaining 117,815 are each labelled by the judge and set beside the assessor's grade. The two label sets are compared in two ways that answer different questions, and only one of them decides whether the judge may be used.

Ordering of systems (the deciding test). Every official participant run of a year is scored twice by nDCG@10, once under the assessors' labels and once under the judge's. The two orderings of the runs are then compared with the Kendall's τ of Section 3.6. This is the question the judge exists to answer: whether replacing the assessors' labels with its own changes which of two systems looks better.

A mean τ of at least 0.7 over the three years was required before the judge could be used. Table 4.3 gives the result. τ is 0.80, 0.87 and 0.84 in the three years, a mean of 0.84, so the judge is used.[1] In 2022, a τ of 0.84 means that about one in thirteen of the 703 pairs formed by 38 runs changes places when the judge's labels replace the assessors'. UMBRELA, validated against fully manual assessment on the TREC 2024 RAG track, reached 0.890 at nDCG@20 [45]; the judge here comes close to that at the shallower cutoff of rank 10.

Agreement on individual answers. The second comparison looks much worse. The Cohen's κ of Section 3.6, over the four grades, is 0.174 across all three years; after binarising at H+M it is 0.381. Taken at face value, 0.174 describes a judge barely better than chance. Two properties of the labels explain it, and neither is noise.

The first is prevalence, the rarity of relevance that Section 4.1 noted. Of the 117,815 pairs, 85.7% are Not relevant and only 6.2% are relevant under H+M. A labeller that read nothing and answered Not relevant every time would still agree with the assessors on 85.7% of pairs.

κ removes that kind of agreement before it counts any. The judge agrees with the assessors on 57.2% of pairs. Given how often each side uses each grade, chance alone would produce 48.2%. So $\kappa = (0.572 - 0.482)/(1 - 0.482) = 0.174$.

[1] The runs are scored with standard nDCG@10 rather than its prime variant. The difference is small here, because both label sets cover the same judged answers except the 0.2% whose judge output could not be parsed.

	Participant	Judged	Kendall's	Cohen's κ	
Edition	runs	pairs	τ	four grades	H+M
ARQMath-1 (2020)	23	39,062	0.802	0.226	0.443
ARQMath-2 (2021)	38	40,179	0.867	0.109	0.379
ARQMath-3 (2022)	38	38,574	0.844	0.185	0.340
All three		117,815	0.838 (mean)	0.174	0.381

Table 4.3 Validation of the LLM relevance judge against the ARQMath assessors, by edition. *Participant runs*: the official runs of that year, each scored by nDCG@10 (normalised discounted cumulative gain over the top ten) under the assessors' labels and under the judge's. *Kendall's* τ: rank correlation between the two resulting orderings of the runs. *Cohen's* κ: agreement on individual (topic, answer) pairs, corrected for the agreement chance would produce; computed over the four relevance grades, and again after binarising at H+M, where High and Medium count as relevant. *Judged pairs*: pairs with both an assessor's grade and a parseable judge label.

The comparison with UMBRELA makes the effect visible. This judge agrees with its assessors more often than UMBRELA does with NIST's, 0.572 against 0.513 over four grades, and still has the lower κ, because 25.8% of the TREC RAG pairs are relevant against 6.2% here.[2] κ values from collections with different relevance rates are therefore not comparable, and this one is not evidence of a weak judge.

The second property is where the disagreements fall. Table 4.4 sets the judge's labels against the assessors'. The judge disagrees on 50,440 pairs, and 36,566 of them (72.5%) sit in a single cell: answers the assessors called Not relevant and the judge called Low. That one confusion accounts for 31.0% of all pairs judged.

It shows in the margins as well. The judge uses Low for 38.1% of pairs where the assessors use it for 8.1%, while at the top of the scale it stays close: Medium 6.0% against 4.0%, High 3.6% against 2.3%.

The error also has a direction. In 44,802 disagreements the judge's grade is above the assessor's, and in only 5,638 it is below. UMBRELA's authors describe the same leniency against NIST's assessors, and their confusion matrix gives 35.0% over against 13.7% under [45, Figure 5]; here it is more one-sided.

[2]UMBRELA's agreement figures are computed here from the confusion matrix its authors publish [45, Figure 5]; they report no κ themselves.

	Judge					
Assessor	0	1	2	3	Total	Share
0 Not relevant	**59,380**	36,566	3,780	1,209	100,935	85.7%
1 Low	1,716	**5,505**	1,624	685	9,530	8.1%
2 Medium	424	2,173	**1,129**	938	4,664	4.0%
3 High	71	696	558	**1,361**	2,686	2.3%
Total	61,591	44,940	7,091	4,193	117,815	
Share	52.3%	38.1%	6.0%	3.6%		

Table 4.4 The assessors' grades (rows) against the judge's grades (columns) for all 117,815 judged pairs of the three ARQMath editions. Grades: 0 Not relevant, 1 Low, 2 Medium, 3 High. The diagonal, set in bold, holds the pairs on which the two agree. The shares say how often each side uses each grade.

Most of it is invisible once grades are binarised: 39,778 disagreements (78.9%) leave the answer on the same side of the H+M threshold, and only 10,662 cross it.

This is also why τ survives a κ this low. The judge's characteristic error is not scattered at random across answers. It promotes Not relevant to Low wherever a non-relevant answer appears, and every participant run retrieves many such answers. A bias that raises every system's score by a similar amount moves the scores together and leaves their order largely intact. κ measures the bias; τ measures what remains of the ordering once the bias has fallen on every system.

The judge is therefore used for one purpose: to order systems against one another. It is not reliable answer by answer. When it calls an answer relevant under H+M, it is right only about one time in three (3,986 of 11,284), and it misses nearly half of the answers the assessors call relevant (3,364 of 7,350). Nor does this validation license reading a score under the judge's labels as a standalone value, whether on its own or beside a score computed from other labels, such as the assessors' or a published system's; such a score means something only next to another score computed from the same judge labels. What remains is how close two systems may be before even their order stops being reliable.

Resolving power

A τ of 0.84 says that most pairs of systems keep their order under the judge's labels. It does not say which pairs do not. Those are, predictably, the close ones,

so any comparison needs to know how close is too close. That is the resolving power of Section 3.6, measured here with the error rate and fuzziness value of Buckley and Voorhees [5].

The calibration uses the 38 official ARQMath-3 participant runs, scored the way this thesis's own systems are scored: each participant run is cut to its top 20 answers, the depth to which the judge labels them; unjudged answers are removed; P′ and nDCG′ are computed at 5, 10 and 20; and the scores are averaged over the 78 topics of Section 4.1. Each participant run is scored twice, under the assessors' labels and under the judge's, over the same 34,781 judged answers. The 66 answers with no parseable judge label are left out of both, so the labels are the only thing that differs between the two scorings. Figure 4.2 plots one score against the other, one point per participant run.

In every panel the points lie close to a straight line (R^2 between 0.95 and 0.97). The slope is between 1.29 and 1.32 for all six measures. The intercept is effectively zero for P′ and 0.07 to 0.08 for nDCG′. The judge's scores are the assessors' scores stretched: a participant run the assessors credit with 0.20 nDCG′@10 is credited with about 0.33 under the judge's labels. A stretch of this kind cannot reorder anything on its own, because multiplying two scores by the same positive number and adding the same constant keeps the larger one larger. Only the scatter around the line can reorder systems, and that is what the error rate measures.

The error rate is the share of pairs of participant runs that the two label sets order differently. Taken over all 703 pairs, nDCG′@10 orders 54 of them differently, or 7.7%. That is close to the one pair in thirteen that the 2022 τ implied, although the two are computed differently. Across the six measures the rate lies between 6.9% and 10.1%.

A fuzziness value treats pairs closer than a threshold as tied rather than ordered. Those are the pairs the judge is most likely to get wrong, so raising the threshold lowers the error rate among the pairs that remain, at the price of ordering fewer of them. The threshold is applied to the difference in judge-label scores, since that is the only difference a later chapter can observe.

At 0.02 the error rate falls below 5% for every measure and cutoff: 3.3%, 4.8% and 4.6% for P′ at 5, 10 and 20, and 3.7%, 3.6% and 4.5% for nDCG′. At nDCG′@10, 603 of the 703 pairs differ by more than 0.02 under the judge's labels; the remaining 100 are treated as ties. In 581 of the 603, the judge's labels and the assessors' labels agree on which run is better. In 22, or 3.6%, they disagree. One measure,

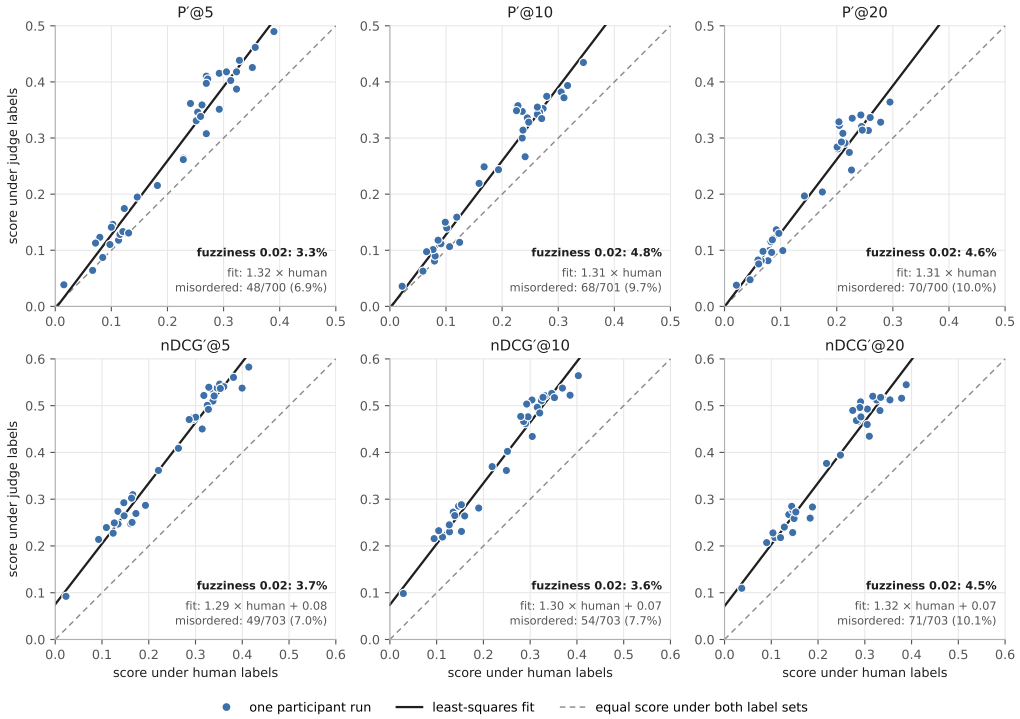


Figure 4.2 Calibration of the LLM relevance judge on ARQMath-3. Each point is one of the 38 official participant runs, scored over its own top 20 answers on the 78 topics of Section 4.1: once under the assessors' labels (horizontal axis) and once under the judge's (vertical axis). Top row: P′@k, precision at k. Bottom row: nDCG′@k, normalised discounted cumulative gain at k. Solid line: least-squares fit. Dashed line: equal score under both label sets. *Misordered*: pairs of runs that the two label sets place in opposite order, out of all 703 pairs. For P′ the count is out of 700 or 701, because a few pairs of runs receive exactly the same P′ under the judge's labels and cannot be ordered at all. *Fuzziness 0.02*: the share of pairs misordered when only pairs whose scores under the judge's labels differ by more than 0.02 are counted, the only pairs later chapters order.

nDCG′@5, would already fall below 5% at 0.01. A single threshold for all six is simpler to apply, and the measure every verdict reads, nDCG′@10, needs 0.02 in any case. For comparison, UMBRELA's authors report that their judge could not tell apart systems closer than about 0.065 nDCG@20 [45]. That figure belongs to a different judge, collection and procedure, and is not used here.

This fixes the rule the retrieval and generation chapters follow. Two systems scored against the same judge labels are ordered only when their scores, reported to three decimals, differ by more than 0.02 on the measure and cutoff in question. A difference of 0.02 or less is reported as indistinguishable. Above the threshold the order is still not certain: it is wrong for fewer than one pair in twenty, not for

none. The scores themselves stay on the judge's inflated scale: a system scoring 0.50 under the judge's labels is ahead of one scoring 0.47, but neither number says what the assessors would have given.

The rule assumes that two systems being compared are scored against the same judge labels. Which answers those labels cover is not settled here, because it depends on what this thesis's systems retrieve; Section 5.2.3 builds that set of labels from their rankings.

Chapter 5

Retrieval

5.1 Overview

This chapter answers the first research question, on retrieval, posed in Section 1.3. Section 5.2 builds the systems it compares and fixes what they are scored against. Section 5.3 answers two threats to those scores before any of them is read: that the formula index does not work at all, and that the ARQMath assessors' labels do not cover what these systems retrieve. Sections 5.4 and 5.5 report and explain the results, and Section 5.6 rules on each hypothesis.

The measures reported are nDCG′ and P′ at cutoffs 5, 10 and 20. nDCG′@10 is the primary measure, and every decision rule that compares answer-retrieval rankings reads it. Where a rule reads more than that, the section applying it says so. The other two cutoffs show the shape of a ranking and decide nothing.

The answer-retrieval values compare retrieval systems with one another; they are not benchmark figures. They are computed against the labels of the relevance judge of Section 4.4, which grades more generously than the ARQMath assessors, so no comparison with published systems for the task is made. Differences are read under that section's rule: a difference of 0.02 or less between the values as reported is indistinguishable.

5.2 Method

Section 5.1 fixed how this chapter's numbers are read. That reading holds only if every ranking was produced as described and every answer-retrieval ranking was scored against the same judgments, so this section states how the rankings were produced and what they are scored against. Chapter 3 defined the formula representations, the three families of retrieval, rank fusion and the prime measures. Chapter 4 placed the indices in the system in Section 4.2, fixed how a ranking is scored in Section 4.3, and validated the judge that labels it in Section 4.4. What remains is what retrieval alone needs. Section 5.2.1 builds the formula indices. Section 5.2.2 turns a topic into queries and fixes the fusions compared. Section 5.2.3 fixes the evaluation pipeline: every answer-retrieval effectiveness figure in Section 5.4, and the retrieval quality Chapter 6 reads, is computed against the judgments it assembles.

5.2.1 Formula representation and indexing

Section 3.2 defined the three tree representations, the path tuples derived from them and the window that bounds a tuple, and Section 3.3 how a sequence of tuples becomes a vector that can be searched. This section states how that was done here: where the trees come from, how the tuples are encoded and embedded, and how the vectors are indexed. The tuples are all the index ever sees of a formula: two formulas can be found similar only through the tuples, and the parts of tuples, that they share.

The implementation adapts the released code of Tangent-CFT [32][1] for parsing trees, extracting tuples and encoding them for fastText, and departs from it in three places. The trees of the collection's formulas are read from the organisers' MathML rather than converted from LaTeX. Each representation has an index of its own rather than contributing to a single vector summed across the three. And the vectors are searched with an approximate nearest-neighbour index, where Tangent-CFT holds every formula vector in memory and compares the query with each of them.

Trees and tuples. The MathML of every formula in the collection is read from the organisers' `latex`, `slt` and `opt _representation_v3` files. These key

[1] https://github.com/BehroozMansouri/TangentCFT

SLT tuples	OPT tuples
(V!x, −, n, −)	(U!eq, O!minus, 0, −)
(V!x, V!y, nn, −)†	(U!eq, O!SUP, 01, −)†
(−, V!y, n, n)	(U!eq, V!x, 00, −)†
(−, N!2, na, n)†	(O!minus, V!x, 0, 0)
(−, =, nn, n)†	(V!x, 0!, 0, 00)
(V!y, N!2, a, nn)	(O!minus, O!SUP, 1, 0)
(N!2, 0!, n, nna)	(O!minus, N!2, 11, 0)†
(V!y, =, n, nn)	(O!minus, V!y, 10, 0)†
(V!y, N!0, nn, nn)†	(O!SUP, V!y, 0, 01)
(=, N!0, n, nnn)	(V!y, 0!, 0, 010)
(N!0, 0!, n, nnnn)	(O!SUP, N!2, 1, 01)
	(N!2, 0!, 0, 011)
	(U!eq, N!0, 0, −)
	(N!0, 0!, 0, 0)

Table 5.1 Every tuple extracted from $x - y^2 = 0$ with a window of two, in the symbol layout tree (SLT) and the operator tree (OPT). Each tuple reads (node, descendant, edge labels on the path between them, path from the root to the first node). Tuples marked † join nodes two edges apart; a window of one produces only the others. `0!` marks the end of a baseline in the SLT and a leaf in the OPT.

each formula by its identifier and record its visual identifier, its post and the post's type, and they are the corrected representations released for ARQMath-3 [31]. A topic's formulas are converted from their LaTeX with LaTeXML [33] when the topic is queried, in both tasks. Each tree is traversed with end-of-baseline markers and a window of two, Tangent-CFT's setting [32]. Table 5.1 gives every tuple this produces for $x - y^2 = 0$: eleven in the SLT and fourteen in the OPT, four of each joining nodes two edges apart. A window of one would give seven and ten.

Encoding. Each tuple becomes one word for fastText. Following Tangent-CFT, a node label is split into its type and its value, so `N!2` contributes the two tokens `N!` and `2`. Edge labels get identifiers of their own, so a relation can never be confused with a symbol. Every token is written as a single Unicode character, with node tokens numbered from 60,000 and edge tokens from 500. A tuple therefore becomes a short string with one character per component, and each of fastText's character n-grams is a run of whole components. The tuples of y^2 and of z^2 at the same position, `(V!y, N!2, a, nn)` and `(V!z, N!2, a, nn)`, are different words, but they share every n-gram that does not include the variable,

Setting	Value
Vector size	300
Character n-grams	length 3 to 6
Objective	skip-gram
Negative samples	20
Context window	5 tuples either side
Epochs	5
Minimum count	1 (every tuple kept)

Table 5.2 fastText settings used to train the tuple embeddings, identical for the symbol layout tree (SLT), the operator tree (OPT) and SLT-TYPE, the SLT with its values erased. The context window counts neighbouring tuples in a formula's tuple sequence, not edges in its tree.

so their vectors lie close together. SLT-TYPE needs no tree of its own: it is the SLT's tuples encoded with every value dropped, so `N!2` contributes `N!` alone.

Position `[3]`, the path from the root, is encoded as part of the word. Tangent-CFT omits it, judging that it may mislead the embedding, particularly in the OPT, where it is only a sequence of argument positions [32]. It is kept here, because without it a pair near the root of one formula and the same pair deep inside another become the same word, so two formulas share vocabulary they do not share structurally. The alternative was not run.

Embedding. One fastText model per representation is trained with gensim [35] over the encoded tuples of the collection's formulas. The settings in Table 5.2 are those Tangent-CFT reports for its SLT and OPT models [32], applied here to all three representations. The one difference is the number of epochs, which the paper does not give: Tangent-CFT's released configuration trains for thirty, and the models here were trained for five, gensim's default.

Indexing. Each representation's vectors are indexed with FAISS [21] as an inverted file with scalar quantisation. The index has 16,384 cells and scans the 256 nearest the query; every vector is stored in half precision, and the cells are learned from a sample of the vectors. The answer-retrieval indices cover the formulas of the answer posts. Each indexed vector keeps the identifiers of its formula and of the answer it occurs in, so a hit on a formula is a hit on that answer. The formula vectors are indexed as they are and searched by squared Euclidean distance. The formula-search index covers every formula in the collection, in questions as well as answers. Each of its vectors is centred by subtracting the average tuple vector

and then scaled to length one, and the query is treated the same way. For vectors of length one, Euclidean distance ranks formulas in the same order as cosine similarity.

The index answers one formula at a time, while a topic is a question holding several. Which of them are sent to the index, and how their hits become one ranking of answers, is the first thing Section 5.2.2 settles.

5.2.2 Retrieval configurations

Section 3.3 described three families of retrieval, each returning a ranked list on a scale of its own, and Section 3.4 a way of combining such lists that never compares their scores. Section 5.2.1 built the formula indices but left one question open: a topic is a question holding several formulas, and a formula index answers one formula at a time. This section settles that question, states how a topic is put to the sparse and dense indices, and fixes the fusions this chapter compares. The same rankings can be combined in more than one way, and whether the way matters is one of the questions Section 5.4.5 answers.

The formula indices are queried by *fanout*: every distinct formula of the topic that is not trivial becomes a query of its own, so the system never has to decide which formula a question is about. A formula is *trivial* when it gives an index nothing to match on. Triviality is judged on the formula's operator tree rather than on its LaTeX string. A formula is trivial when no tuples can be extracted from it, as happens for an extraction artefact such as `a)`, or when its tree holds no operator, function or matrix node and at most one distinct variable, number or constant. A bare set name such as $\mathbb{R}$ is therefore trivial, while x^n is not, since its tree has the superscript operator `O!SUP` that Table 5.1 shows for y^2. A test on string length would get both wrong, keeping a long variable name and discarding a short equation. The test is applied to the operator tree whichever index is queried, so all three formula indices receive the same query formulas, and a topic whose formulas are all trivial is queried with the one that has the most operator-tree tuples. Fanout and the triviality filter are the design choices of this thesis, held fixed in every experiment.

Each query formula returns the 1,000 formulas closest to it, and every one of them counts towards the answer it appears in. A formula counts for more the higher it is ranked: it adds $1/(60 + r)$ to its answer's score, where r is its rank in

its query's list, which is the reciprocal-rank weighting of Section 3.4. The answers are then ranked by their totals, so an answer containing several formulas close to the question's rises above an answer containing only one.

The sparse and dense indices are queried with the topic's text, its title and question as parsed in Section 4.1, with the formulas left in their LaTeX. The sparse index is built with Pyserini [27], the Python interface to Anserini, which is built on Lucene, and scored by BM25 with that toolkit's default parameters, $k_1 = 0.9$ and $b = 0.4$. The dense index holds one vector per answer post from `multi-qa-mpnet-base-dot-v1`, a bi-encoder from the sentence-transformers library [36], built on the MPNet language model [42] and fine-tuned on question–answer pairs. The topic text is encoded with the same model and compared by dot product with every stored vector, so the dense index is searched exhaustively where the formula indices are searched approximately. The encoder is used as published, with no further training on mathematical text.

Each index returns at most 1,000 answers per topic, so every topic yields five ranked lists of answer posts: BM25, dense, and one for each formula representation. Each list is stored as a run, and every fusion reads the stored runs rather than querying an index again, so a fused list can be rebuilt from the runs it combines. A retrieval *configuration* is a choice of which of these rankings are used and, where there is more than one, how they are combined. Fifteen configurations are reported: each of the five rankings on its own, and the ten fusions of Table 5.3.

Every fusion is reciprocal rank fusion with $k = 60$ over whole rankings of answers, so a ranking contributes to an answer's score at most once; no score is normalised and no ranking is weighted. The three formula rankings can enter a fusion in two ways, and the difference is the point. Fused with one another, SLT, OPT and SLT-TYPE give a single ranking, the *formula block*. In a *blocked* fusion the formula block enters beside the text rankings as one input; in a *flat* fusion the three formula rankings enter separately, each on the same footing as BM25 and dense. Since reciprocal rank fusion weighs every input equally whatever it retrieves, a flat fusion of all five hands three of the five votes to three views of the same formulas, where a blocked fusion hands them one of three. A blocked configuration therefore combines at three levels: the hits of each query formula within one representation, the three representations within the block, and the block with the text rankings.

The ten fusions in Table 5.3 answer the two sub-questions with few runs rather than sweeping every combination, and they fall into five groups. Dense retrieval is the text side in every pair, as the stronger of the two text indices in the single-index results of Section 5.4.2, so that the text side is held fixed and any difference between the three pairs comes from the representation. The last group removes one index from a full system, a *leave-one-out* ablation, and the index removed is the weakest of its kind on its own: BM25 of the two text indices, and SLT-TYPE of the three formula indices, as those same single-index results show. Reciprocal rank fusion gives a weak ranking the same vote as a strong one, so if an index costs a fused system more than it adds, the weakest is the likeliest to. SLT-TYPE is removed from the flat fusion only: there its ranking has a full vote, as much as dense retrieval, while inside the blocked fusion it is only one of the three rankings that make up the block.

At formula search the formula block is the only configuration, run over the collection-wide index of Section 5.2.1. A formula-search topic is a single formula, so there is nothing to select: each representation returns the 1,000 formula instances nearest the topic formula, the three lists are fused by reciprocal rank fusion with $k = 60$, and the result is collapsed to the organisers' visual identifiers, keeping the highest-ranked instance of each, before it is cut to 1,000. Instances sharing the topic formula's own visual identifier are left out.

Every answer-retrieval configuration thus ends in one ranked list of answers per topic, fifteen rankings of each topic in all. What those rankings are scored against — which of their answers carry a relevance label, from whom, and to what depth — is the question Section 5.2.3 settles.

5.2.3 Evaluation protocol

Section 3.5 showed that the ideal nDCG divides by is computed from a topic's judgments, not from the ranking being scored, so two nDCG values can be compared only when both are computed against the same judgments. The fifteen rankings of Section 5.2.2 therefore need one set of judgments between them, and the organisers' cannot be it: their judgments cover what the 2022 participants retrieved, not what these rankings retrieve. Section 4.4 validated a language model as a relevance judge: its labels put systems in the same order as the assessors' labels do, so it can supply judgments for answers the assessors never saw. Which

Configuration	Rankings fused
Text only — the system the formula indices are added to	
BM25 + dense	BM25, dense
One formula index added — the text side held fixed, so the representation is the only thing that changes	
dense + SLT	dense, SLT
dense + OPT	dense, OPT
dense + SLT-TYPE	dense, SLT-TYPE
Formula only — the three formula rankings with no text	
formula block	SLT, OPT, SLT-TYPE
All three added — the same five rankings, differing only in whether formula evidence enters as one input or as three	
blocked	BM25, dense, formula block
flat	BM25, dense, SLT, OPT, SLT-TYPE
One index removed — if a system is worse without an index, that index was contributing	
blocked, −BM25	dense, formula block
flat, −BM25	dense, SLT, OPT, SLT-TYPE
flat, −SLT-TYPE	BM25, dense, SLT, OPT

Table 5.3 The ten fused configurations on answer retrieval, each a reciprocal rank fusion ($k = 60$) of the rankings listed. *SLT*: symbol layout tree. *OPT*: operator tree. *SLT-TYPE*: the SLT with its values erased, node types only. The *formula block* is the three formula rankings fused into one; a *blocked* fusion takes it as a single input, a *flat* fusion takes the three separately. A minus sign marks the index removed.

answers those are is settled here. This section defines the set of answers the judge labels, how a ranking is scored against that set, and what formula search is scored against instead. The set fixes the ideal every answer-retrieval ranking is measured against.

Only the head of a ranking is labelled, because only the head is used: the generator of Section 4.2 receives the top k answers of a ranking and nothing below them. Each ranking is labelled to rank 20. That covers the cutoffs of 5 and 10 and leaves room to show how a ranking continues below them. Twenty is therefore

also the deepest cutoff reported: a measure reading further would reach answers the ranking was never labelled for.

The labels are gathered by the pooling of Section 3.1, with the fifteen configurations in place of the participants' runs and one depth, 20, for all of them. For each of the 78 topics, the top 20 answers of every configuration are merged and deduplicated, and each answer in the result is graded by the judge. That set of labels is the *judge pool*. No assessor's grade enters it, not even for an answer the organisers judged, so that every label in it comes from one judge applying one scale. Fifteen rankings of twenty answers could put at most 300 answers into a topic's pool; because the rankings overlap, each topic's pool holds between 81 and 116 answers, 7,532 in all across the 78 topics. The judge's output for eight further answers could not be parsed, and those answers are left unjudged.

A ranking is scored against the judge pool after being cut to its top 20 and then condensed, as the prime measures require. Inside that top 20 every answer carries a label but for any of those eight, so the results of Section 5.4 report no hole rate.

Because every configuration is scored against the same judge pool, each topic has one ideal, shared by all fifteen, and their nDCG′ values can be set beside one another. The ideal is drawn from the pool alone, so it holds only what the fifteen configurations retrieved: a relevant answer that none of them placed in its top 20 raises no topic's ideal. That is the second reason, beside the judge's generous grading noted in Section 5.1, why the answer-retrieval values compare configurations with one another and are not benchmark figures.

Formula search is scored as the organisers scored it, because Section 5.3 sets the formula block beside a published system. The judgments are theirs: 11,538 judged pairs of a topic and a visually distinct formula over 76 topics, of which 1,215 are High, 1,600 Medium, 1,990 Low and 6,733 Not relevant. The measures are ARQMath's own for the task, nDCG′ and MAP′ over the whole ranking with P′@10 beside them. MAP′ appears here and in none of the answer-retrieval results, because it belongs to the measure set the published figures are given in. No judge labels are used there: the judge grades answers to a question, not formulas.

The organisers' judgments for answer retrieval are not set aside. Section 5.5 measures how far a topic's formulas appear inside the answers the assessors judged relevant and inside those they judged not relevant; it computes no retrieval measure, so the judge pool does not enter it. And Section 5.3 scores the

same fifteen rankings against the judgments of Table 4.1 as the organisers scored their participants, condensing the full ranking before the cutoff, and asks how much of these rankings those judgments leave unjudged and what that does to the scores they give — the second of the two threats that section takes up.

5.3 Guards

Every score in this chapter rests on two things that could fail before any of them is read. The formula index could simply not work, in which case nothing built on it means anything. And the ARQMath assessors' labels could fail to cover what these systems retrieve, in which case the scores measure the reach of a judgment pool these systems never contributed to rather than the quality of their retrieval. Both are answerable without any answer-retrieval result, which is why they come first. Neither rules on a hypothesis.

Formula search. The formula index is this thesis's own construction, and every answer-retrieval result depends on it: if it does not rank formulas sensibly, each configuration that uses it is measuring a broken component. That is answerable on its own, on the task the index was built for. Formula search asks for exactly what the formula block does — given a formula, find the formulas most like it — and it judges relevance on the formula itself rather than on the post containing it. The block is measured here against a published formula-search system, and what it reaches becomes the level against which the answer-retrieval results are read.

The configuration is the formula block of Section 5.2.2, run on the collection-wide formula index of Section 5.2.1 and scored as Section 5.2.3 fixes: the organisers' judgments over their 76 assessed topics, and the measures they publish for the task. A comparison on those measures carries no resolving-power limit — the 0.02 of Section 4.4 was measured for the judge's labels on answer retrieval and does not transfer to the assessors' labels on another task. What the figures below can support is therefore placement, whether the block performs comparably with a published system, and not an ordering between the two.

The comparator is TangentCFT2, DPRL's automatic formula-search run [28], and it is the bar to use because it is the same idea: the formula stack of Section 5.2.1 is an implementation of Tangent-CFT, so a gap between the two is a gap between implementations and not between approaches. Three differences

System	Pooled	nDCG′	MAP′	P′@10	Hole@10
TangentCFT2	15	0.641	0.419	0.534	0.026
Formula block (this thesis)	—	0.615	0.420	0.592	0.355

Table 5.4 Formula search on the 76 assessed ARQMath-3 topics. *Formula block*: this thesis's symbol layout tree (SLT), operator tree (OPT) and SLT-TYPE — the SLT with its values erased — rankings fused into one. *Pooled*: the depth to which a run's formulas entered the judgment pool, a dash marking a run that did not contribute to it. The prime measures nDCG′ (normalised discounted cumulative gain) and MAP′ (mean average precision) read the whole ranking and P′@10 the top ten, each after formulas carrying no judgment are dropped. *Hole@10*: the share of the top ten carrying no judgment. TangentCFT2's scores are those published in the lab overview, Table 4 [31]; its hole rate is measured here from its submitted run.

remain, and each of them favours the comparator. DPRL index four representations, an SLT and an OPT and a type-only variant of each, where this thesis indexes three. Their fusion carries the similarity score into the reciprocal-rank numerator, where the fusion here reads ranks alone. And their primary run, TangentCFT2TED, reranks by tree edit distance, a step with no counterpart here.

The two perform comparably (Table 5.4). The formula block is ahead by 0.058 on P′@10, level on MAP′ at the precision the lab reports, and behind by 0.026 on nDCG′.

The two ranking-wide measures split for a reason that recurs in this chapter. The block retrieves 1,715 of the 2,815 formulas these topics' judgments mark relevant under H+M binarisation, three fifths of them; the remainder were surfaced only by systems that contributed to the pool. nDCG′ divides by an ideal built from every judged formula, and condensing a ranking removes its unjudged hits without shrinking that ideal (Section 3.5), so a relevant formula the block never retrieved counts against it. P′@10 reads the head of the ranking alone, and there the block is ahead. The 0.026 therefore comes from relevant formulas the block never retrieved at all, not from how it orders the ones it did.

Non-participation does not decide the table. TangentCFT2 helped build the judgments that score it and the formula block did not, which is the standing objection to any comparison made afterwards. What that membership bought is measurable, and at rank 10 it is 0.026 against 0.355: nearly every formula in TangentCFT2's top ten carries a grade, against roughly two thirds of the block's. Three things bound what that gap can do.

The first is that pool membership guarantees little depth. TangentCFT2 is DPRL's alternate run, so the organisers judged its first fifteen visually distinct formulas per topic; below rank fifteen it meets unjudged formulas like anything else.

The second is that the judgments covering a ranking here are rich in relevant formulas. Of the 11,538 judged pairs of Section 5.2.3, 2,815 are relevant under H+M binarisation, close to a quarter, against 2,943 of 34,847 for answer retrieval (Table 4.1), under a tenth. Condensing discards the unjudged hits, so what is left to score is drawn from a set in which relevant formulas are common rather than rare.

The third is that one judgment covers more than the formula that earned it. Instances sharing a symbol layout tree were clustered and each cluster judged once, taking the highest grade any of its instances received [29]. Judging fifteen visually distinct formulas therefore covers every occurrence of those fifteen anywhere in the collection, and the block, which ranks structure and collapses its hits onto the organisers' own identifiers, lands inside clusters that were already judged without having had to find the instances a participant found. That is why it leaves 0.355 of its top ten unjudged here, where the same three representations, fused the same way, leave 0.751 on answer retrieval (Table 5.5).

On the task the model it implements was built for, then, the formula block retrieves at the level of a published system. That settles the component and nothing else: it says the index is not broken, not that the same representations will find an answer post, which is what Section 5.4 measures. And the 0.751 just quoted is the second threat in miniature — how much of these rankings the assessors' labels leave unjudged, and what those labels make of the rest, is what the next part measures.

Hole rates under the ARQMath labels. Section 5.2.3 scored the fifteen rankings against labels a language model supplied, and that choice needs its evidence. The ARQMath judgments were assembled from what the 2022 participants retrieved, so they cover these rankings only where these rankings agree with those. This part measures how far that goes: how much of each ranking the assessors judged, and what their labels make of the part that is left. The same ranking scores 0.029 or 0.218 depending only on how an unjudged answer is counted, and neither number is a statement about retrieval.

Configuration	Hole@10	nDCG@10	nDCG′@10
BM25	0.386	0.205	0.270
Dense	0.349	0.271	0.369
SLT	0.772	0.090	0.283
OPT	0.800	0.077	0.281
SLT-TYPE	0.917	0.029	0.218
Formula block	0.751	0.095	0.278
BM25 + dense	0.227	0.306	0.364
Dense + SLT	0.405	0.280	0.395
Dense + OPT	0.421	0.261	0.384
Dense + SLT-TYPE	0.518	0.217	0.384
Blocked	0.238	0.330	0.399
Blocked, − BM25	0.386	0.283	0.397
Flat	0.569	0.185	0.357
Flat, − BM25	0.678	0.140	0.343
Flat, − SLT-TYPE	0.503	0.225	0.373

Table 5.5 How much of each configuration's ranking the ARQMath assessors judged, and what their labels make of it. Macro average over the 78 official ARQMath-3 answer-retrieval topics, scored against the organisers' judgments alone; a leading minus names the index removed. *Hole@10*: the share of the top ten answers carrying no ARQMath judgment. *nDCG@10*: normalised discounted cumulative gain at rank 10, an unjudged answer counted as not relevant. *nDCG′@10*: the same, with unjudged answers removed before the cutoff. *SLT*: symbol layout tree; *OPT*: operator tree; *SLT-TYPE*: the SLT with its values erased.

Table 5.5 gives the hole rate of Section 3.5 at rank 10 for each configuration, beside nDCG at the same cutoff computed two ways from the same judgments, over the 78 topics of Section 4.1. The spread of the first column is the thing to note. BM25 leaves 0.386 of its top ten unjudged and the dense index 0.349; both retrieve the kind of text answer the 2022 participants retrieved. The four formula-only configurations leave between 0.751 and 0.917, so that two or three answers in ten carry a grade, and for SLT-TYPE fewer than one. The nine fused configurations run from 0.227 to 0.678: fusing the two text rankings leaves less unjudged than either alone, and the more formula rankings a fusion carries the more it leaves.

The two remaining columns differ only in what they do with a hole. Standard nDCG counts every unjudged answer as not relevant and scores the top ten as re-

trieved. The prime variant, the organisers' own, removes unjudged answers from the ranking first, so the ten answers it scores are the ten highest-ranked answers carrying a grade — for a configuration at 0.917, drawn from far below rank 10. At these hole rates that difference is most of the score: the four formula-only configurations read 0.029 to 0.095 in the first column and 0.218 to 0.283 in the second. Neither figure describes the ranking a reader would see. The first punishes a configuration for retrieving outside the pool, and the second, as Section 3.5 noted, is optimistic for the same reason.

The two columns then disagree about the same three systems. Under the prime measure SLT reaches 0.283 and OPT 0.281 against BM25's 0.270, and the formula indices are level with the sparse text index. Under the standard measure BM25 reads 0.205 against SLT's 0.090 and OPT's 0.077, and it is far ahead. Same rankings, same judgments, one convention apart. Neither ordering can be reported as a retrieval result, and the question of whether the formula indices reach BM25 is Section 5.4.2's, not this table's.

The information is not in these judgments, and no choice of measure puts it there. That is the gap the judge pool of Section 5.2.3 fills, by labelling the whole of every ranking's top twenty rather than the part the 2022 pool happens to cover. Section 5.4 reports against it.

5.4 Results

5.4.1 Overview

Five experiments answer the retrieval question, each changing one thing about what the fusion receives. They run in the order the argument needs: what each index does alone, then one formula index added to text retrieval, then all three, then the arrangement in which they enter, and last whether each of them earns its place.

5.4.2 Single-index retrieval

No hypothesis is decided here. Every comparison that follows adds a formula index to a text index or removes one from a combination, and each is read against what these indexes do on their own, so those levels are established first.

	nDCG′			P′		
Configuration	@5	@10	@20	@5	@10	@20
BM25	0.331	0.338	0.360	0.144	0.121	0.101
Dense	**0.508**	**0.557**	**0.629**	**0.287**	**0.287**	**0.258**
SLT	0.216	0.230	0.259	0.085	0.082	0.074
OPT	0.221	0.229	0.251	0.090	0.068	0.065
SLT-TYPE	0.104	0.104	0.119	0.054	0.045	0.037

Table 5.6 Each retrieval index used on its own. ARQMath-3 answer retrieval, the 78 official topics of Section 4.1, judge-pool labels. *SLT*: symbol layout tree. *OPT*: operator tree. *SLT-TYPE*: the symbol layout tree with its values erased. *nDCG′*: prime normalised discounted cumulative gain. *P′*: prime precision. The highest value in each column is set in bold.

Dense retrieval leads, at 0.557 nDCG′@10 and 0.287 P′@10 over the 78 official topics. BM25 reaches 0.338 and 0.121, and the three formula indexes fall below both — 0.230, 0.229 and 0.104 at nDCG′@10, so none of them reaches even the weaker text index. Among the configurations built without formula evidence, dense retrieval alone at 0.557 also stays ahead of the two text rankings fused at 0.539, which makes it the strongest text-only retrieval under the rule of Section 1.3 and the reference bar for every comparison in this section.

The three formula indexes do not separate cleanly from one another. SLT reaches 0.230 and OPT 0.229 at nDCG′@10, a difference of 0.001 against a resolving-power limit of 0.02, and their order is not stable across cutoffs: OPT is ahead at rank 5 and SLT at rank 20. SLT-TYPE is last in every cell of the table, at 0.104, less than half of either. Erasing the values from a symbol layout tree and keeping only its node types therefore costs most of what the representation carries.

A ranking this far below another can still hold something that other one lacks. Section 5.4.3 fuses one of them with dense retrieval to find out.

5.4.3 Adding one formula index to text retrieval

The two families fail in different ways. Formula retrieval places answers containing the question's formulas at the head of its ranking and cannot order them; dense retrieval orders answers well and reads no formula structure at all. Fusion is the standard answer to two rankings that fail in unrelated ways. A ranking that

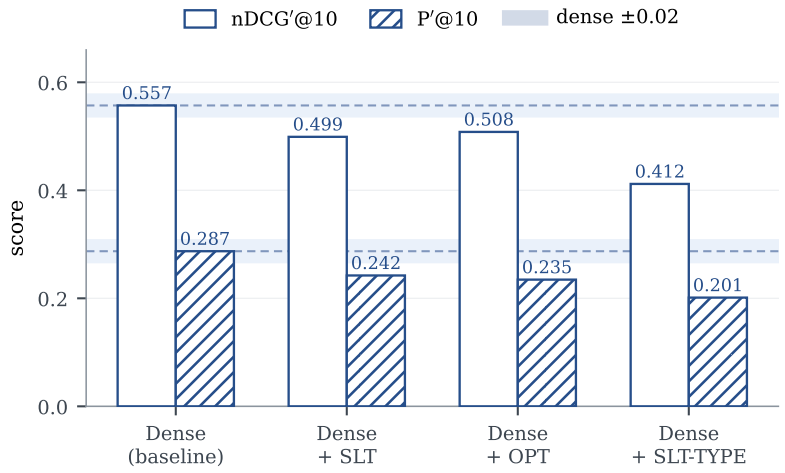


Figure 5.1 Dense retrieval against each configuration that adds one formula index to it. The dashed lines mark dense retrieval's score and the bands the 0.02 resolving-power limit of Section 4.4, within which two scores are indistinguishable. ARQMath-3 answer retrieval, the 78 official topics of Section 4.1, judge-pool labels. *SLT*: symbol layout tree. *OPT*: operator tree. *SLT-TYPE*: the symbol layout tree with its values erased. *nDCG′@10*: prime normalised discounted cumulative gain at rank 10. *P′@10*: prime precision at rank 10.

scores poorly on its own can still earn a place in one, provided what it finds is not what the other finds. Hypothesis H1.1 makes that expectation testable: adding any one formula index to the strongest text-only retrieval improves on it. It holds only if all three pairs are ahead of dense retrieval on both nDCG′@10 and P′@10, where a gap of 0.02 or less is not ahead.

None of the three is ahead (Figure 5.1). Dense retrieval with OPT reaches 0.508 at nDCG′@10, with SLT 0.499 and with SLT-TYPE 0.412, against dense retrieval's 0.557; on P′@10 they read 0.235, 0.242 and 0.201 against 0.287. All six gaps are wider than 0.02. The highest of the three is dense retrieval with OPT, though it stands 0.009 above dense retrieval with SLT, which is inside the limit. How far a pair falls below dense retrieval tracks the index added to it. The pair with SLT-TYPE, last of the five indexes on its own, sits 0.145 below dense retrieval at nDCG′@10; the pairs with SLT and OPT sit 0.058 and 0.049 below it.

Adding one formula index to dense retrieval moves it down, not up. That is not what H1.1 predicted, and Section 5.6 rules on it. Whether three together behave differently is a separate question, and Section 5.4.4 asks it.

5.4.4 Adding all three formula indexes

Each of the three representations reads a formula differently. A symbol layout tree records how it is written, an operator tree what it computes, and SLT-TYPE its shape with the values gone, so an answer that one of them misses may be one another finds. Three rankings that fail on different queries could cover for each other in a way no single pair can. Hypothesis H1.2 states that expectation in two clauses: the three-index system improves on the strongest text-only retrieval, and it improves on adding one. The system it names is dense retrieval with SLT, OPT and SLT-TYPE, each ranking entering the fusion on its own; how they enter is held fixed here and varied in Section 5.4.5, so that one thing changes at a time. Both clauses must hold on nDCG′@10 and P′@10, each gap beyond 0.02.

The first clause fails, and not narrowly (Figure 5.2). The four-ranking system reaches 0.322 nDCG′@10 and 0.124 P′@10, against dense retrieval's 0.557 and 0.287. It is 0.235 behind on the first measure and 0.163 behind on the second, eight times the limit in each case.

The second fails with it. Against the highest of the three pairs of Section 5.4.3, dense retrieval with OPT at 0.508 and 0.235, the four-ranking system is 0.186 and 0.111 behind. Adding three formula rankings therefore leaves the system further back than adding one did, and adding one had already left it behind dense retrieval alone. More formula evidence makes the result worse, not better, and the fusion rule is a candidate explanation: reciprocal rank fusion gives the top of every ranking the same weight and has no way of knowing which ranking is the better one, as Section 3.4 sets out, so each formula ranking added is another vote of equal standing.

Neither clause holds, and Section 5.6 rules on H1.2. If the vote count is what does the damage, then arranging the same three rankings so that they hold fewer votes should undo some of it, and Section 5.4.5 tests exactly that.

5.4.5 How the formula rankings enter the fusion

The four-ranking system of Section 5.4.4 gave each of the three formula rankings a vote of its own. Nothing about those rankings, or about the fusion rule, is changed here. Only the way they enter it is. Section 5.2.2 fixed two arrangements for that. The blocked system merges the three formula rankings into a single input before the final fusion, so formula evidence holds one vote of three there.

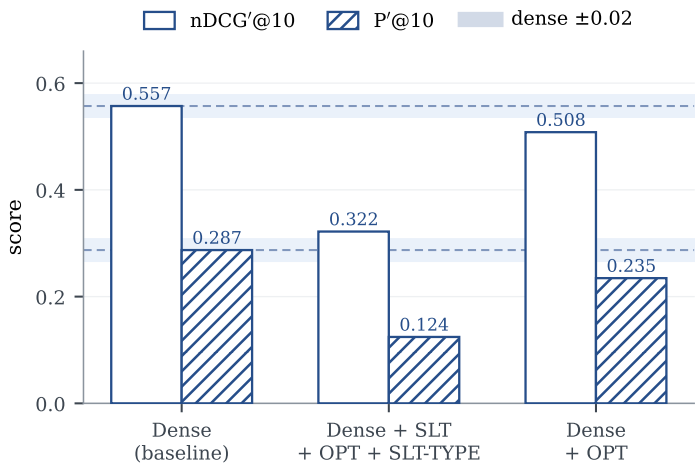


Figure 5.2 Dense retrieval against dense retrieval with all three formula rankings, each voting on its own, and against the highest of the single-index configurations. The dashed lines mark dense retrieval's score and the bands the 0.02 resolving-power limit of Section 4.4. ARQMath-3 answer retrieval, the 78 official topics of Section 4.1, judge-pool labels. *SLT*: symbol layout tree. *OPT*: operator tree. *SLT-TYPE*: the symbol layout tree with its values erased. *nDCG′@10*: prime normalised discounted cumulative gain at rank 10. *P′@10*: prime precision at rank 10.

The flat system lets all five rankings enter the final fusion directly, so formula evidence holds three votes of five. Hypothesis H1.3 predicts that blocking beats flat fusion. The prediction rests on consistency: blocking must be ahead in every cell of the comparison between the two full systems — nDCG′ and P′ at 5, 10 and 20. Six cells, and one going the other way fails it.

Blocking is ahead in all six (Figure 5.3). At nDCG′@10 the blocked system reaches 0.551 against the flat system's 0.374. The margins run from 0.081 on P′@20 to 0.179 on nDCG′@20, every one of them clear of 0.02, and the ordering holds at every cutoff and on both measures.

Blocking could work in either of two ways.

The first is that merging makes a better ranking. The three formula rankings fused with one another might be stronger than any of them alone, in which case the final fusion is simply receiving something better than it received before.

The second is that merging changes nothing about the formula evidence and only changes how many votes it holds. Reciprocal rank fusion gives every input an equal vote, as Section 3.4 sets out, so the two vote counts above are the whole of the difference between the arrangements. Every configuration in this chapter that

took in formula evidence ended up below dense retrieval alone. On this reading, then, blocking helps by letting less of that evidence through, not by making it better.

Merging does improve the formula ranking on its own terms. Fused together, the three reach 0.261 nDCG′@10 (Table 5.7), above SLT's 0.230 and OPT's 0.229 in Table 5.6 by more than 0.02. But no formula evidence is created by merging them. The flat system is given the same three rankings, so whatever the merged ranking finds, the flat system's inputs find as well. What separates the two arrangements is therefore not what the final fusion is shown. It is how much of the final ranking that evidence is allowed to decide.

	nDCG′			P′		
Configuration	@5	@10	@20	@5	@10	@20
Formula block	0.244	0.261	0.285	0.100	0.091	0.078

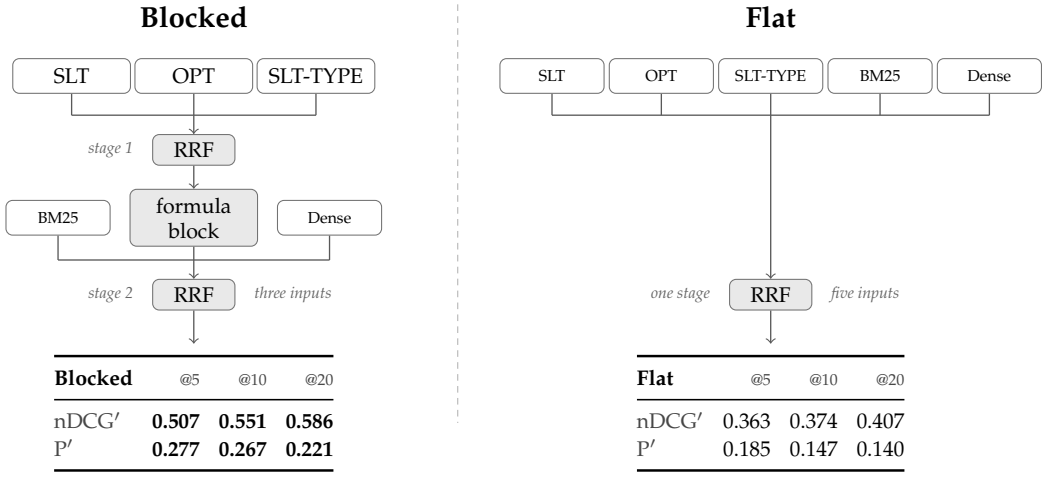
Table 5.7 The three formula rankings fused with one another — the *formula block* — scored on its own. ARQMath-3 answer retrieval, the 78 official topics of Section 4.1, judgepool labels; *nDCG′* prime normalised discounted cumulative gain and *P′* prime precision, unjudged answers removed before scoring.

The hole rates of Section 5.3 show that weighting at work, since the text rankings return mostly answers the ARQMath assessors judged and the formula rankings mostly answers they did not: 0.569 of the flat system's top ten carries no judgment, against 0.238 of the blocked system's. With three votes of five the formula rankings fill much of the flat list; with one vote of three they fill little of the blocked one.

Blocking beats flat fusion in every cell measured, which is what H1.3 predicted, and Section 5.6 rules on it. That formula evidence does less harm when it holds fewer votes leaves one thing open: whether the harm is spread evenly across the three rankings or concentrated in one of them. Section 5.4.6 separates those.

5.4.6 Removing an index

If a weak ranking costs a fusion more than it contributes, taking it out should leave the fusion better off. That runs against the assumption a multi-index system is built on, which is that each index brings something the others do not.



Blocked	@5	@10	@20
nDCG′	**0.507**	**0.551**	**0.586**
P′	**0.277**	**0.267**	**0.221**

Flat	@5	@10	@20
nDCG′	0.363	0.374	0.407
P′	0.185	0.147	0.140

Figure 5.3 Blocked against flat fusion. Both fuse the same five rankings and differ only in how the three formula rankings enter: *blocked* merges them into one input before the final fusion, *flat* lets all five enter it directly. Blocking is ahead in all six cells, by 0.081 to 0.179. ARQMath-3 answer retrieval, the 78 official topics of Section 4.1, judge-pool labels. *RRF*: reciprocal rank fusion. *SLT*: symbol layout tree. *OPT*: operator tree. *SLT-TYPE*: the symbol layout tree with its values erased. *nDCG′*: prime normalised discounted cumulative gain. *P′*: prime precision.

Hypothesis H1.4 predicts that every index earns its place, so that removing any one of them makes the system worse. It is rejected if any removal improves the system at nDCG′@10. Three ablations test it — BM25 out of each full system, and SLT-TYPE out of the flat fusion, which is the one arrangement where its ranking holds a vote of its own.

Two removals behave as predicted (Table 5.8). Without BM25 the blocked system falls from 0.551 to 0.522 at nDCG′@10 and the flat system from 0.374 to 0.322, both by more than 0.02, so the weaker text index contributes to each of them despite what it scores alone. The flat system is worse without BM25 in all six cells. The blocked system's loss shows on nDCG′ at ranks 10 and 20; in the remaining four cells the two are indistinguishable.

The third runs the other way. Without SLT-TYPE the flat fusion rises from 0.374 to 0.416 at nDCG′@10, and the gain clears 0.02 in five of six cells, the exception being P′@5 at 0.015. SLT-TYPE was last of the five indexes on its own in Section 5.4.2, and the fusion is better off without it. Under a rule that weights every ranking's top positions alike whatever that ranking is worth, a weak index does not merely fail to help; it pushes better candidates down the fused list.

	nDCG′			P′		
Configuration	@5	@10	@20	@5	@10	@20
Blocked	0.507	**0.551**	**0.586**	0.277	0.267	0.221
Blocked, − BM25	0.510	0.522	0.551	0.297	0.256	0.214
Flat	0.363	0.374	0.407	0.185	0.147	0.140
Flat, − BM25	0.313	0.322	0.349	0.151	0.124	0.109
Flat, − SLT-TYPE	0.407	0.416	0.449	0.200	0.183	0.162

Table 5.8 The two full systems and the three configurations that remove one index from them, the removed index marked by a leading minus. *Blocked*: the three formula rankings merged into one input. *Flat*: each entering the fusion on its own. ARQMath-3 answer retrieval, the 78 official topics of Section 4.1, judge-pool labels. *SLT-TYPE*: the symbol layout tree with its values erased. *nDCG′*: prime normalised discounted cumulative gain. *P′*: prime precision. Where the two leading configurations differ by more than the 0.02 resolving-power limit of Section 4.4, the higher value is set in bold; in the remaining four columns they are indistinguishable and neither is marked.

One index is therefore not earning its place, and Section 5.6 rules on H1.4. The removal does not explain the whole of Section 5.4.5's gap, though: without SLT-TYPE the flat system reaches 0.416 and stays below the blocked system's 0.551 in all six cells, by 0.059 to 0.137, even though the blocked system still contains SLT-TYPE.

What none of these comparisons explains is why formula evidence should hurt the retrieval it is added to. That is a question about the signal rather than about the fusion, and Section 5.5 takes it up by looking at where a topic's formulas actually sit.

5.5 Where the formula signal lives

Section 5.3 measured the formula block on the task its design was aimed at, formula search, and found it performing comparably with a published system. Section 5.4 measured the same three representations on answer retrieval, and every configuration they joined scored below the text retrieval they were added to. The index is not broken and the fusion is not misarranged; what changes between the two results is what the task counts as relevant. This section takes that up. It decides no hypothesis, and explains a result Section 5.4 has already established.

Nothing is retrieved for this measurement. It reads the organisers' judgments for the 78 official topics of Section 4.1 directly, the assessors' labels rather than the judge's, since the question is what the assessors counted as relevant. Every formula on both sides is converted to the tuples of Section 5.2.1, and a pair of formulas, one from the topic and one from an answer, is scored by *containment*: the fraction of the topic formula's tuples that appear in the answer's, so that one is a complete match. The best-matching pair is kept for each answer, and the topic's formulas are selected by the fanout of Section 5.2.2, so that what is measured is what the formula indexes are given.

Relevant answers are compared against two controls, a hard one and an easy one, each matched in number to the topic's relevant set. The hard control is the non-relevant judged answers of the same topic, which isolates relevance from mere topical similarity. The easy control is answers drawn at random from the collection. A topic's per-answer containment values are then collapsed three ways: the mean over all answers, the mean of the top five, and the single best answer. This shows how the signal behaves as the group narrows. The narrow end is the end an evaluation measure reads, since nDCG′@10 and P′@10 see only the first few answers a ranking returns.

Averaged over all of a topic's answers, containment separates the two classes (Figure 5.4). Relevant answers contain 0.377 of the query formula's OPT tuples against 0.238 for the same topic's non-relevant answers and 0.095 for random ones, and they lead the hard control on 70 of the 78 topics. Those non-relevant answers were read against this question and rejected, so they are about it too, and they still contain less of its formulas. Being about the question is not what produces the difference. Narrow the group and the separation erodes at every step: at the single best-matching answer relevant answers reach 0.829 against 0.719, and the lead falls to 30 topics, on 13 of which the non-relevant answer contains more and on 35 of which the two are exactly equal. SLT behaves as OPT does, and SLT-TYPE separates worst of the three at the narrow end, leading on 17.

This is why formula retrieval under fanout cannot match retrieval over text. Fanout queries with every non-trivial formula in the topic, so almost every plausible candidate scores highly and therefore scores alike: the right neighbourhood is retrieved, and nothing is left with which to rank inside it. The relevance criteria then work against the index. A post that only restates the question is non-relevant

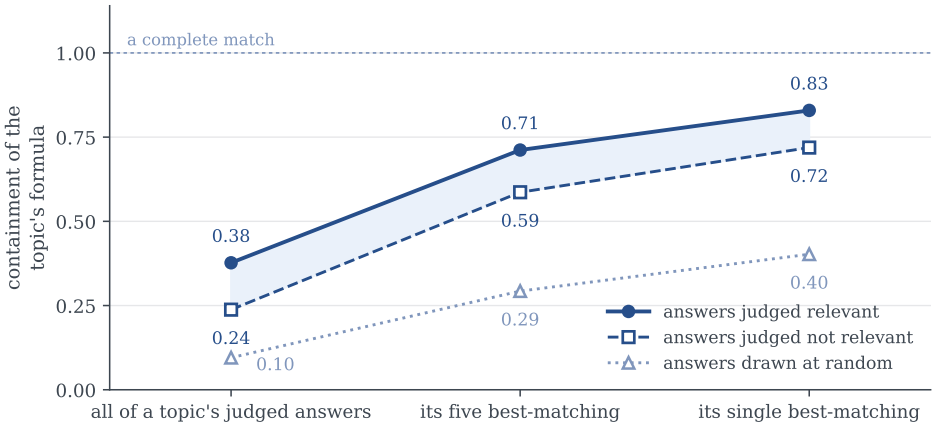


Figure 5.4 Containment of a topic's formulas in its judged answers, under the *OPT* (operator tree) representation of Section 5.2.1, over the 78 official topics of Section 4.1 and scored against the organisers' own judgments. *Containment*: the fraction of a topic formula's path tuples that also appear in an answer's formula, so that 1.00 is a complete match. Each point is a mean over the 78 topics, and the three columns narrow the answers a topic contributes, from all of its judged answers to the single one that matches best. The non-relevant and random groups are matched in number to the topic's relevant set.

by definition (Section 3.1). A restatement carries the question's formulas verbatim. So the closest structural match a formula index can find is often the answer the criteria reject. Formula search grades the other way: its top level is calibrated against exact structural match, which is close to what containment measures. The words around a formula still separate an answer that derives a result from a comment that restates the question using the same expression, and that is what the text rankings read. Section 5.6 takes this up with the results.

5.6 What this settles about RQ1

Section 5.4 ran five experiments, and four of them decide a hypothesis. Formula evidence that holds up at formula search adds nothing to answer retrieval that a dense retriever has not already found. More of it makes the result worse rather than better. The arrangement that lets the least of it through is the best system this chapter builds.

Adding one formula index. H1.1 predicted that each formula index, added to the strongest text-only retrieval, improves answer retrieval over that retrieval

alone. Section 5.4.2 resolves that dense retrieval on its own scores 0.557 nDCG′@10 over the 78 official topics. None of the three pairs reaches it — 0.508 with OPT, 0.499 with SLT and 0.412 with SLT-TYPE, and 0.235, 0.242 and 0.201 on P′@10 against 0.287 — and all six gaps are wider than 0.02 (Figure 5.1). H1.1 is not supported, and the answer to RQ1.1 is no.

Adding all three. H1.2 predicted that adding all three formula indexes improves on the strongest text-only retrieval, and improves on adding one. Dense retrieval with the three formula rankings each voting on its own reaches 0.322 nDCG′@10 and 0.124 P′@10, behind dense retrieval alone by 0.235 and 0.163 and behind the best of the pairs, dense retrieval with OPT, by 0.186 and 0.111 (Figure 5.2). Both clauses fail and H1.2 is not supported. Three formula rankings leave the system further back than one did.

How the formula rankings are merged. H1.3 predicted that blocking beats flat fusion — the same five rankings, with formula evidence holding one vote of three rather than three of five. Blocking is ahead in all six cells the rule reads, by 0.081 to 0.179, and at the primary measure 0.551 against 0.374 (Figure 5.3). H1.3 is supported.

Whether every index earns its place. H1.4 predicted that every index contributes, so that removing any one of them makes the fused system worse. Removing BM25 costs both full systems. Removing SLT-TYPE lifts flat fusion from 0.374 to 0.416 nDCG′@10, past 0.02 there and in five of the six cells (Table 5.8), so one index does not earn its place. The reciprocal rank fusion of Section 3.4 gives every input the same vote however weak that input is: blocking limits how many votes the formula rankings hold, and removing SLT-TYPE takes away the weakest of them. Blocking is supported, and the hypothesis that every index earns its place is not. The two verdicts carry one finding: how the rankings are merged and which of them are chosen both matter.

The best multi-index retriever. Among the configurations built from more than one index, the blocked system is the highest at the primary measure, at 0.551 nDCG′@10: ahead of every flat arrangement and of the blocked system without BM25 at 0.522. It is therefore the best multi-index retriever this chapter produces, and it is the retriever Chapter 6 gives a generator. It does not beat text-only retrieval. Against dense retrieval it differs by 0.006 on nDCG′@10 and 0.020 on P′@10, neither past the 0.02 of Section 4.4, so the two are indistinguishable on both reported measures. Section 5.4.5 shows what brings it that close:

with formula evidence holding one vote of three, less of it reaches the final ranking. Blocking earns its place by keeping that evidence from doing harm, not by supplying what the text rankings lack.

RQ1 as a whole. Adding structure-aware formula indexes to text-only retrieval does not improve answer retrieval. One formula index leaves the system 0.049 to 0.145 behind dense retrieval at the primary measure, depending on which one is added. All three, each voting on its own, leave it 0.235 behind. Only the blocked arrangement of all five rankings draws level with dense retrieval. How they are combined does govern the result, and it is the whole of what the fused systems differ by. The failure is not a property of the index. Section 5.3 measured the same three representations at formula search, where they perform comparably with a published system (Table 5.4), and Section 5.5 located the difference in what each task counts as relevant: a topic's formulas are contained in the answers the assessors judged relevant more than in those they judged not relevant, leading on 70 of the 78 topics, but among the best-matching answers, where a ranking is decided, that lead falls to 30 (Figure 5.4). Formula matching retrieves the right neighbourhood and cannot order inside it, and answer retrieval is graded on whether a post resolves the question.

Chapter 6 asks whether retrieval differences of these sizes reach the answer a generator writes. Between the multi-index retriever and dense retrieval the difference is too small to resolve; between either of them and any formula representation on its own it is large.

Chapter 6

Generation

6.1 Overview

This chapter answers the second research question, on generation, and the third, a sanity check on the second: whether its answers survive a change of generator. Section 6.2 states how the answers were produced and how they were scored. Section 6.3 reports three experiments: how answers generated from the multi-index retriever's context compare with answers generated from no context and from the context of text-only retrieval; whether answer quality tracks retrieval quality across the fifteen retrieval configurations of Section 5.2.2; and whether both results hold as the generator grows. The chapter closes by ruling on each hypothesis.

The retriever this chapter gives the generator is the one Section 5.6 selected, the blocked multi-index system, and the retrieval it is set against is the strongest text-only system, dense retrieval on its own. Both were chosen there by reading the retrieval results, and this chapter takes them as given.

Two measures are reported, and they answer different questions. Factual-correctness precision (FC-P) is the quality measure: the share of an answer's claims that the reference supports. Faithfulness measures attribution rather than quality — the share of an answer's claims that the retrieved context supplied — so a low value on a *correct* answer means the generator already knew the answer, not that the answer is poor. Faithfulness needs a context to be computed against, and generation without retrieval therefore has no value for it.

Retrieval quality enters as the primary measure of Chapter 5, nDCG′@10 under the labels of the relevance judge of Section 4.4. Differences between generation scores are read as the values are reported, with no statistical test and no equivalent limit: the 0.02 of Section 5.1 is a property of the relevance judge and governs retrieval scores alone.

6.2 Method

Section 3.7 defined retrieval-augmented generation and the two measures this chapter reports, and Section 4.2 placed the generator at the end of the pipeline: the highest-ranked answers of a ranking become its context, and one fixed prompt turns the question and that context into an answer. This section states what was generated, from which contexts, and how the answers were scored. There is no query rewriting, no reranking, no agentic loop and no adjustment of the prompt from one set of answers to the next.

What was generated

Three sets of answers, all on the 78 topics of Section 4.1. The first receives no context at all: the generator is given the question alone and answers from its own knowledge. This is the *floor*, the level any retrieved context has to improve on. The second set comes from the fifteen retrieval configurations of Section 5.2.2, each supplying the five highest-ranked answers of its own ranking as the context for that topic. The third repeats part of that work at two larger generators, for the sanity check the third research question asks: the floor and five of the fifteen configurations — the two text indexes, their fusion, the formula block and the blocked system — are generated again at 3 and at 7 billion parameters. Everything else is held fixed, so what separates any two sets of answers is the context the generator was given, and in the third set the size of the generator as well.

A context of five answers is a design choice rather than a measured optimum. It matches the size of the reference the answers are scored against, and it is held fixed everywhere, so that no comparison mixes a difference in how much context was supplied with a difference in which retrieval supplied it. No sweep over that size is run.

```
[SYSTEM]
You are a mathematics question-answering assistant. Answer the
question as accurately and completely as you can, and write
all mathematics in LaTeX. Retrieved context may be provided
below; when it is, use it wherever it helps, and prefer it
over your own recollection where the two conflict. You may
also draw on your own knowledge, whether or not retrieved
context is provided. Give the reasoning needed to justify the
answer, without restating the question or padding.

[USER]
Retrieved context:
[1] <first retrieved answer>

[2] <second retrieved answer>

   ...

[5] <fifth retrieved answer>

Question:
<the topic's title and question body>

Answer:
```

Figure 6.1 The answer-generation prompt. One template serves every set of answers in this chapter. The retrieved answers are numbered in the order the ranking returns them and stand here as placeholders. When no context is supplied the *Retrieved context* block is absent and the rest of the template is unchanged, which is how the answers without context — the floor — are generated. The system message is a single line in the template and is wrapped here to fit the page.

The prompt

Figure 6.1 shows it, and one template serves everything in this chapter. It presents the retrieved answers, asks for the question to be answered, and permits the generator its own knowledge alongside the context while asking it to prefer the context where the two conflict. It instructs no refusal, so a weak context still yields an answer rather than a decline. When there is no context the retrieved-context block disappears and the rest of the template is unchanged, which is how the floor is generated: the floor and every retrieval configuration render from the same template. That is what makes Section 6.3 a comparison of contexts.

Setting	Value
Generator	`Qwen/Qwen2.5-1.5B-Instruct`
Generators, sanity check	`Qwen/Qwen2.5-3B-Instruct`
	`Qwen/Qwen2.5-7B-Instruct`
Precision	bfloat16
Temperature	0.0
Top-p	1.0
Maximum new tokens	2,048
Seed	42
Runs per setting	1
Context size	5 answers
Scoring framework	RAGAS 0.4.3
Scoring model	`gemini-3.5-flash-lite`, temperature 0
Claim decomposition	atomicity high, coverage high

Table 6.1 Generation and scoring settings. Every answer in this chapter is produced and scored with these settings; the generator changes only where its size is the thing being varied. *Atomicity* and *coverage* govern how finely a text is decomposed into claims (Section 3.7). The scoring model is the one the scoring framework uses for its own judgements, and is not the relevance judge of Section 4.4.

The generator

Table 6.1 lists the settings. Decoding is greedy: the temperature is zero, so the generator always takes the most probable next token, and answers are produced one prompt at a time. Each setting was generated once, and greedy decoding leaves nothing to average — with the temperature at zero and the seed fixed, a repeat of the same setting returns the same answers. The limit on new tokens is 2,048, and no answer reached it, so every answer was scored whole rather than broken off in mid-derivation.

The reference

Factual correctness needs a reference, and Section 3.7 left what it is built from to this section. It is built from the assessors' judgments alone, never from the relevance judge's labels, which would otherwise grade the pipeline against labels its own judge supplied. A mathematics question usually admits several valid routes to the same result, so a single answer taken as the reference would mark a

correct answer wrong for having taken another route. The reference is therefore the union of up to five judged answers for the topic: High-labelled answers first, filled from Medium where a topic has fewer than five, and ordered within a label by the community score the collection records. Seventy-four of the 78 topics have five or more such answers and take the full five; the remaining four take the two, three or four they have. No topic is scored without a reference.

The measures as computed

Both are computed with RAGAS 0.4.3, whose own language model performs every judgement. It is the same model the relevance judge of Section 4.4 uses, doing a different task as a different instrument; nothing in that section validates it.

Faithfulness is computed in two steps. The model first rewrites the answer into standalone statements, each understandable without the others. It then judges each statement against the retrieved context taken as one body of text and returns a verdict with a reason. The score is the share of statements it can infer from the context. Where an answer yields no statements at all the score is undefined rather than zero, which happens once across the runs reported here.

FC-P follows the same shape with the reference in place of the context. The model decomposes the answer into claims and checks each against the reference; the reference itself is never decomposed, so the score is the share of the answer's claims that the reference supports. Both parameters Section 3.7 names are set high — atomicity, so that a sentence is split into its indivisible claims, and coverage, so that the detail of each survives the split — and the setting is the same for every answer in this chapter. Across the answers reported here the decomposition yields six claims on average, with a median of six and a range from one to twenty-two. The framework rounds each factual-correctness score to two decimals. Faithfulness and FC-P decompose the same answer separately, so the two scores are not fractions of one set of claims.

Read together, the two measures say where an answer came from. Because the generator is never required to confine itself to the context, a correct answer may be its own work or the context's, and the pair distinguishes the two: a high FC-P with a low faithfulness is a correct answer the generator largely wrote from what it already knew, while a high FC-P with a high faithfulness is a correct answer the context supplied. What the pair cannot supply is a ceiling for faithfulness.

Since no answer here was written under an instruction to use the context alone, faithfulness is read as a share of what the answer says and never as a distance from an attainable maximum.

Relating answer quality to retrieval quality

Two hypotheses read an association across configurations rather than a comparison between two sets of answers. Retrieval quality is nDCG′@10 under the judge labels of Section 5.2.3, the primary measure of Chapter 5, over the same fifteen configurations. The association is reported as the Kendall's τ of Section 3.6, computed over all 105 pairs of those fifteen. It is computed on the means before rounding, and it is reported for its sign and its size. No statistical test is applied to it.

The sanity check reads the same coefficient over the five configurations generated at all three sizes — ten pairs — and reads them at 1.5 billion parameters as well, so that the three sizes are compared on one instrument. Those three values may be set beside one another; none of them may be set beside the coefficient over fifteen configurations, which is computed from ten times as many pairs.

6.3 Results

6.3.1 Overview

Three experiments answer the questions this chapter asks, and they run in the order the argument needs. The first two change what the generator is given and hold everything else fixed: whether a retrieved context reaches the answer at all, and then whether a better context reaches it further. The third changes the generator rather than the context, repeating both at three and at seven billion parameters.

6.3.2 What the retrieved context changes

A retrieval system that ranks better does not thereby produce a better answer. The generator may ignore what it is given, or use it and be carried wrong by it, and nothing in Chapter 5 says which happens. This section puts three settings beside

one another on the 1.5-billion-parameter generator: no context at all, the context of the strongest text-only retrieval, and the context of the multi-index retriever. One template serves all three, so what separates them is the context.

Two hypotheses read these answers. Hypothesis H2.1 predicts that the multi-index retriever's context makes answers more factually correct than generation without retrieval, and holds only if the blocked system is ahead of the floor on FC-P. Hypothesis H2.2 predicts that the same context makes answers both more factually correct and more faithful than the strongest text-only retrieval's, and holds only if the blocked system is ahead of dense retrieval on both measures. Neither reads a threshold: the values are read as they are reported.

Against no context. The floor reaches 0.170 FC-P over the 78 topics of Section 4.1 and the blocked system 0.206, ahead by 0.036 (Table 6.2). It is ahead on 33 of the 78 topics, behind on 23 and level on 22; the scoring framework rounds each factual-correctness score to two decimals, which is why so many topics come out level. Section 6.4 rules on H2.1.

The floor is not the bottom of the field, though. Ten of the fifteen configurations of Section 5.2.2 stand above it on FC-P and five below — OPT, SLT, the formula block, dense with SLT-TYPE, and dense with the three formula rankings each voting separately. A retrieved context is therefore not an improvement in itself. Below some level of retrieval quality the answers built from a context are worse than answers written from none.

The one configuration that breaks the pattern marks what drives it. SLT-TYPE alone is the weakest retrieval of the fifteen and its answers are nonetheless not dragged below the floor, at 0.196, while their faithfulness is the lowest in the chapter at 0.049 — almost nothing they say comes from the context (Figure 6.2). Each of the five that fall below the floor draws on its context more than that. A context weak enough to be ignored costs the generator nothing, and a context used but wrong costs it the answer.

Against the strongest text-only retrieval. Section 5.6 found the two systems indistinguishable as retrieval, 0.551 against dense retrieval's 0.557 on nDCG′@10. Their answers are not indistinguishable. On faithfulness the blocked system is ahead, 0.189 against 0.171, and ahead on 37 topics against 33; on FC-P it is behind, 0.206 against 0.241, and behind on 39 topics against 24. H2.2 needs both, so the correctness half decides it, and Section 6.4 rules on it.

Context	nDCG′@10	Faithfulness	FC-P
No context — the floor	—	—	0.170
Dense retrieval	0.557	0.171	**0.241**
Blocked multi-index	0.551	**0.189**	0.206

Table 6.2 Answers generated without a context, from dense retrieval's context, and from the blocked multi-index retriever's. *nDCG′@10*: prime normalised discounted cumulative gain at rank 10, the retrieval quality of Chapter 5. *FC-P*: factual-correctness precision. Faithfulness is undefined without a context. The 78 topics of Section 4.1, five retrieved answers as context, 1.5-billion-parameter generator. The highest value in each of the two answer-quality columns is set in bold.

The two measures answer different questions, as Section 6.1 set out, and here they answer in opposite directions. More of what the generator says under the multi-index context is traceable to that context, and less of what it says is right. The multi-index context is used more and helps less.

Two comparisons read two points each, and the field has fifteen. Whether where a context sits in the retrieval ordering predicts anything about the answers built from it is the question the next section asks of all fifteen at once.

6.3.3 Retrieval quality and answer quality

The two comparisons of Section 6.3.2 each read a pair of systems, and a pair cannot say whether the relation holds in general: two systems may differ for reasons that have nothing to do with how well either retrieved. The fifteen configurations can answer it, because they retrieve at very different levels — from 0.104 to 0.557 nDCG′@10.

Two hypotheses put the question, one for each measure. Hypothesis H2.3 predicts that faithfulness rises with the quality of the retrieved context: the better the context, the more of the answer comes from it. Hypothesis H2.4 predicts the same of factual correctness. Each is rejected if its measure does not rise with retrieval quality across the fifteen. Every pair enters the Kendall's τ, and no resolving power is applied to it: the 0.02 of Section 4.4 sets no pair aside as too close to order.

Both measures rise with retrieval quality (Figure 6.2). Ordering the fifteen by retrieval quality and again by faithfulness leaves the two orderings concordant on 96 of the 105 pairs, $\tau = 0.829$. Ordering them by factual correctness instead

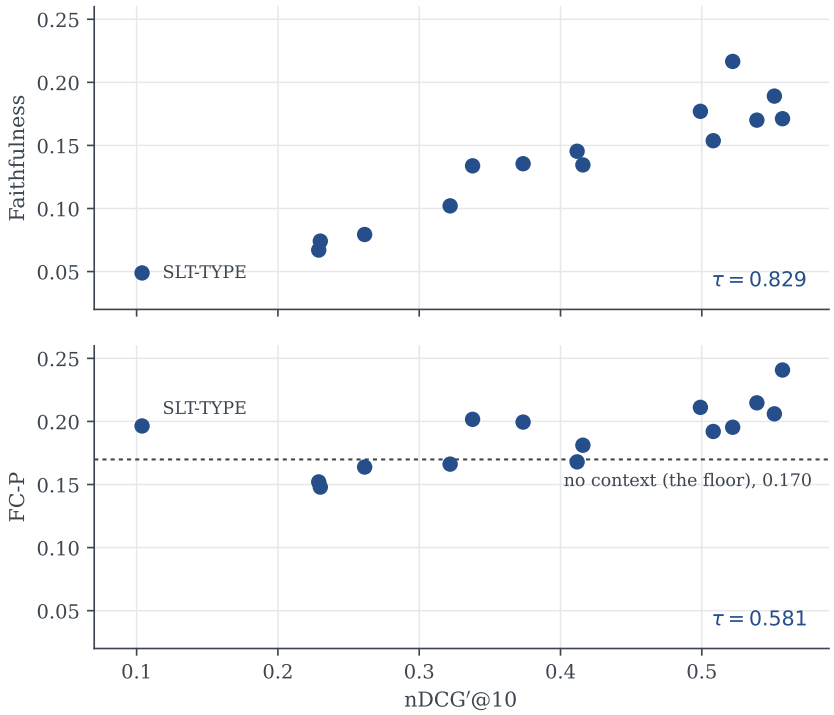


Figure 6.2 Faithfulness and factual-correctness precision (FC-P) of the generated answers, against the retrieval quality of the configuration that supplied the context, over the fifteen configurations of Section 5.2.2. Kendall's τ in each panel, over all 105 pairs. The dashed line marks generation without retrieval, which has an FC-P but no faithfulness and so appears in the lower panel alone. SLT-TYPE is named in both panels as the configuration the surrounding text argues from. *nDCG′@10*: prime normalised discounted cumulative gain at rank 10, under the judge labels. The 78 topics of Section 4.1, five retrieved answers as context, 1.5-billion-parameter generator.

leaves 83 of 105 concordant, $\tau = 0.581$. The closing section of this chapter rules on H2.3 and H2.4.

Read together, the two say more than either does alone. The generator draws on the context it is given, and the better that context, the more of the answer comes from it: only 9 of the 105 pairs run the other way. Correctness rises too, and less evenly, with 22 pairs out of order. The chain from retrieval to the answer has two links and they are not equally tight. A better context is more likely to be used, and being used is not the same as being right.

Where correctness runs backwards is worth naming. Eight of its 22 discordant pairs involve SLT-TYPE alone. It retrieves worst of the fifteen and its faithfulness is the lowest in the chapter, 0.049, so the generator hardly touches what it supplies

		1.5B		3B		7B	
Context	nDCG′@10	Faith.	FC-P	Faith.	FC-P	Faith.	FC-P
No context — the floor	—	—	0.170	—	0.201	—	0.246
formula block	0.261	0.079	0.164	0.072	0.214	0.086	0.229
BM25	0.338	0.134	0.202	0.150	0.214	0.162	0.243
BM25 + dense	0.539	0.170	0.215	0.174	0.204	**0.225**	0.272
blocked	0.551	**0.189**	0.206	**0.214**	0.214	0.197	**0.281**
dense	0.557	0.171	**0.241**	0.189	**0.313**	0.221	0.277

Table 6.3 The floor and the five configurations generated at all three generator sizes, ordered by retrieval quality. *Faith.*: faithfulness, undefined without a context. *FC-P*: factual-correctness precision. *nDCG′@10*: prime normalised discounted cumulative gain at rank 10, under the judge labels. *1.5B*, *3B*, *7B*: generators of 1.5, 3 and 7 billion parameters. The 78 topics of Section 4.1, five retrieved answers as context. The highest value in each faithfulness and FC-P column is set in bold; the nDCG′@10 column orders the rows and is not marked.

and writes much the same answer it would write from no context at all. At 0.196 that answer stands above eight of the fourteen configurations that retrieve better than it does (Figure 6.2). A context the generator does not use cannot pull its answers down to the level of the retrieval behind it.

Both orderings are read on the smallest generator, on the same fifteen contexts throughout. Whether either survives a larger one is the last question this chapter asks.

6.3.4 The size of the generator

Everything so far is read on a 1.5-billion-parameter generator, the smallest of the three. A model that size has less of its own to bring, so a retrieved context has more room to matter to it, and a larger model may need it less. The third research question asks whether the answers to the second survive that change. It is a sanity check on those answers rather than a question about generators in their own right.

The floor and five of the fifteen configurations — the two text indexes, their fusion, the formula block and the blocked system — were generated again at 3 and at 7 billion parameters, with the prompt, the context size, the decoding and the topics all held fixed (Table 6.3). Hypothesis H3.1 predicts that the conclusions

of the second research question are not specific to the smallest generator, in two clauses. The first asks that each of the three comparisons of Section 6.3.2 — the blocked system against the floor on FC-P, and against dense retrieval on FC-P and on faithfulness — runs in the same direction at the larger sizes as it does at the smallest. The second asks that both measures still rise with retrieval quality there.

The three comparisons at each size. All three keep their direction at 3 billion parameters; two of them reverse at 7 billion. Against the floor, the blocked system stays ahead on FC-P throughout: 0.206 against 0.170 at 1.5 billion parameters, 0.214 against 0.201 at 3 billion and 0.281 against 0.246 at 7 billion. That is the direction Section 6.3.2 found, and both larger generators keep it. Against dense retrieval the blocked system is behind on FC-P at the two smaller sizes, 0.206 against 0.241 and 0.214 against 0.313, and ahead at the largest, 0.281 against 0.277; on faithfulness it runs the other way, ahead at the two smaller sizes, 0.189 against 0.171 and 0.214 against 0.189, and behind at the largest, 0.197 against 0.221. Both of these match Section 6.3.2 at 3 billion parameters and reverse at 7 billion. The two reversals run in opposite senses: under the largest generator the blocked system's context produces the more factually correct answers of the two, and supplies less of what those answers say.

Retrieval quality at each size. Over the five conditions common to all three sizes, faithfulness rises with retrieval quality at every size: $\tau = 0.800$ at 1.5 billion parameters, 0.800 at 3 billion and 0.600 at 7 billion. Factual correctness rises at the smallest and the largest, $\tau = 0.800$ in both, and at 3 billion it does not, $\tau = 0.000$, with five of the ten pairs in one order and five in the other. The value at 1.5 billion is a reference point rather than a test — the clause reads the two larger sizes, and computing the same τ over the same five conditions puts all three sizes on one instrument. All three are read over ten pairs where Section 6.3.3 reads 105, so a single pair changing places moves one of them by 0.200; they may be set beside each other and beside nothing else. Section 6.4 rules on H3.1.

What the ladder shows about generators. One thing more is worth recording, and it decides nothing. The floor's FC-P rises at every step in generator size: 0.170 at 1.5 billion parameters, 0.201 at 3 billion and 0.246 at 7 billion. At 1.5 billion it stands above one of the five conditions, at 3 billion above none, and at 7 billion above two — BM25 at 0.243 and the formula block at 0.229, against its own 0.246. Answers built from those two contexts are less factually correct than answers

built from no context at all. The larger the generator, the more of the answer it already holds, and the better a retrieved context has to be before it adds anything.

6.4 What this settles about RQ2 and RQ3

Section 6.3 ran three experiments, and each of them decides a hypothesis. A retrieved context does reach the answer, and it reaches it in proportion to how good the retrieval was. What it reaches is how much of the answer the generator borrows, and borrowing more is not the same as being right more often.

Whether a retrieved context helps at all. H2.1 predicted that the multi-index retriever's context makes answers more factually correct than generation without retrieval. It does: 0.206 FC-P against the floor's 0.170 over the 78 topics of Section 4.1, ahead on 33 of them and behind on 23. H2.1 is supported. The result does not generalise to retrieval as such, though. Five of the fifteen configurations fall below the floor, and the configuration that escapes that pattern explains it: SLT-TYPE retrieves worst of all fifteen, its answers draw less on their context than any other, and they are not dragged below the floor with it (Figure 6.2). A context the generator ignores costs it nothing. A context it draws on costs it the answer when the answer is not there.

The multi-index retriever against text-only retrieval. H2.2 predicted that its context makes answers both more factually correct and more faithful than the strongest text-only retrieval's. Only the second holds: 0.189 faithfulness against dense retrieval's 0.171, and 0.206 FC-P against 0.241. H2.2 is not supported, and the answer to RQ2.1 is yes to its first half and no to its second. The two measures moving in opposite directions is the finding, and Chapter 5 has already named its cause. The formula rankings put posts in front of the generator that share the question's formulas, and Section 5.5 established that formula containment identifies the right neighbourhood and cannot order inside it. The generator does what the ranking did: it treats those posts as pertinent and draws on them. Formula similarity attracts the generator's attention without carrying the answer.

Whether better retrieval reaches the answer. H2.3 and H2.4 predicted that faithfulness and factual correctness rise with retrieval quality across the fifteen configurations. Both do, and both are supported, so the answer to RQ2.2 is yes. They do not rise equally: the ordering by retrieval quality and the ordering by faithfulness agree on 96 of the 105 pairs, $\tau = 0.829$, and by factual correctness on

83, $\tau = 0.581$. The chain from a retrieval score to a correct answer has two links and only the first is tight. How much of a context gets used follows from how pertinent it looks, which is close to what a retrieval score measures. Whether the answer comes out right depends on whether that context actually held the answer, which a retrieval score measures much less well.

Whether any of it survives a larger generator. H3.1 predicted that the answers to RQ2 are not specific to the smallest generator. They are not robust to it. Every comparison holds its direction at 3 billion parameters and two of the three reverse at 7 billion, and factual correctness stops rising with retrieval quality at 3 billion altogether. H3.1 is not supported and the answer to RQ3 is no — though what fails is narrower than that verdict sounds, since RQ2.1 replicates cleanly at the middle size. The reason is visible in the floor: a generator that already holds more of the answer raises the bar a context has to clear, and at 7 billion parameters two of the five contexts fall below it.

RQ2 and RQ3 as a whole. One fact sits underneath all of it. The highest faithfulness anywhere in this chapter is 0.225: in every configuration, at every generator size, most of what an answer says comes from the generator rather than from what was retrieved. Retrieval is a minority contributor to the text, and the rest follows from that. It is why the multi-index retriever's context can be drawn on more heavily and still produce less correct answers. It is why a clean ordering at the retrieval end arrives at the answer as a loose one. And it is why the whole effect thins as the generator grows and has more of its own to offer. Retrieval reaches the answer; it does not write much of it.

Chapter 7 takes up what the two chapters say together, and what this thesis could not measure.

Chapter 7

Concluding remarks

7.1 Summary

This thesis built a retrieval-augmented system for mathematical question answering and asked what structure-aware formula retrieval contributes to it. Three indexes over formula trees were built beside a sparse and a dense text index, and the rankings they produce were merged in ten arrangements. Those arrangements were scored on answer retrieval over the 78 topics of the ARQMath-3 collection, under the labels of a relevance judge validated before it was relied upon. The same fifteen configurations then supplied contexts to a generator, and its answers were scored for how much of each came from its context and how much of each was correct.

Formula evidence did not improve answer retrieval. Each formula index added to dense retrieval left the system behind dense retrieval alone, and all three added together left it further behind still. How the rankings were merged did decide the outcome. Collapsing the three formula rankings into one vote before fusing beat letting each of them vote separately, 0.551 against 0.374, and that arrangement is the best multi-index retriever this thesis produced. It does not beat text-only retrieval. At 0.551 against dense retrieval's 0.557 the two are indistinguishable on both reported measures.

A retrieved context did reach the answer. The multi-index context made answers more factually correct than generation without retrieval, 0.206 against 0.170, and both faithfulness and factual correctness rose with retrieval quality across the fifteen configurations. Against the strongest text-only retrieval the

same context was drawn on more heavily and produced less correct answers: 0.189 faithfulness against 0.171, and 0.206 factual correctness against 0.241. Both readings are taken at 1.5 billion parameters, and both hold at 3 billion. The gains thin as the generator grows: at 7 billion parameters a generator holds more of the answer itself, so a context has more to clear before it helps, and some of these comparisons run the other way.

Three things follow from the two chapters read together, and none of them can be stated inside either one.

Formula overlap misleads a ranker and a generator alike. Section 5.5 located why formula evidence does not help answer retrieval. A topic's formulas are contained in the answers the assessors judged relevant more than in those they judged not relevant, leading on 70 of the 78 topics; among the best-matching answers, where a ranking is decided, that lead falls to 30. Formula matching finds the right neighbourhood and cannot order inside it. Chapter 6 shows the generator doing the same thing to the same posts. Under the multi-index retriever's context more of the answer is traceable to that context than under the strongest text-only retrieval's, 0.189 against 0.171, and less of what the answer says is correct, 0.206 against 0.241. The ranking treats formula overlap as pertinence and places those posts in front of the generator; the generator treats them as pertinent and draws on them. One signal misleads two stages in the same direction, and it reaches the second stage through the first.

What a retrieval score predicts about an answer. Across the fifteen configurations of Section 5.2.2 the ordering by the primary measure and the ordering by faithfulness are concordant on 96 of the 105 pairs, and the ordering by factual correctness on 83. A retrieval score predicts how much of an answer will come from its context more closely than it predicts whether that answer will be right. Where the measure resolves nothing, the answers still differ: the blocked system and dense retrieval are 0.006 apart on the primary measure, inside the 0.02 of Section 4.4, and their answers separate on both generation measures, in opposite directions. The relation also does not hold evenly at the weak end. SLT-TYPE retrieves worst of the fifteen, at 0.104, and its answers are not dragged below generation without a retrieved context, while five configurations that retrieve better than it are. Almost nothing of its answers comes from its context, at 0.049 faithfulness, and a context that is not drawn on cannot carry an answer wrong. What decides the cost of a weak context is how heavily it is used, not how weak it is.

What formula evidence alone amounts to. Section 5.6 found that blocking earns its place by keeping formula evidence from doing harm rather than by supplying what the text rankings lack. Chapter 6 prices the evidence that still reaches the generator. The three configurations built from formula indexes alone are among the five whose answers fall below generation without a retrieved context. A well-matched formula, delivered without the prose that resolves the question, is not a usable context at either stage: the ranking cannot order inside the neighbourhood it finds, and the generator writes a worse answer from it than from nothing at all.

Each of these readings rests on conditions measured once, on one collection, and scored by instruments this thesis validated rather than assumed. Section 7.2 states what that leaves unsettled.

7.2 Limitations

1. **The relevance judge.** Every retrieval score in Chapter 5 is computed from labels a language model assigned. Section 4.4 shows that the judge orders systems as the assessors do, which is what this thesis reads it for, and that it is lenient about individual labels, agreeing with them at a Cohen's κ of 0.174. The scores therefore say which system is better, not how good any of them is. Two scores differing by 0.02 or less say nothing at all, and that limit was measured on the 38 official participant runs of one year. Whether it holds on another collection is untested.

2. **The generation reference.** Factual correctness is measured against the union of up to five judged answers for each topic. A mathematics question usually has more than one route to the result. Those five answers may take several of those routes or may all take the same one, and either way they do not cover every route. An answer that is right by a route none of them takes counts as unsupported.

3. **No human reading of the generated answers.** Both generation measures are computed by the evaluation framework's own language model. Section 4.4 validates the relevance judge, and Section 6.2 says that this is a different instrument doing a different task. Nothing here shows that these factual-

correctness scores agree with a mathematician reading the same answers. Nothing here judges an answer as a whole either. Both measures work over the separate claims an answer is split into, so no result in Chapter 6 says that an answer is right or that it is wrong.

4. **Factual correctness and the length of an answer.** Factual-correctness precision is the share of an answer's claims that the reference supports. A longer answer makes more claims, and each further claim is one the reference may not cover. A low score can therefore come from length rather than from error. Nothing holds the answers to a common length, and the length they run to differs from one condition to the next. Some of what separates two conditions may be how much each answer said.

5. **Answering efficiency was never measured.** How fast a system answers is part of whether it can be used. Every run reported here was made offline, in batch. No latency was timed and no query throughput was counted, so nothing here says how the system would behave under a query load.

7.3 Future work

1. **Tuning the answer-generation prompt.** This thesis presents one prompt and holds it fixed, so that what separates its conditions is the context and nothing else. That leaves the prompt itself unexamined. The highest faithfulness anywhere in Chapter 6 is 0.225, so most of every answer comes from the generator rather than from what was retrieved. A prompt that directs the generator more firmly at its context would raise that share, and whether it also raises factual correctness is the question to put to it.

2. **Choosing which formula to query with.** The formula indexes here are queried with every non-trivial formula a topic contains. Section 5.5 shows what that costs. With so many query formulas, almost every plausible answer matches one of them, and the ranking has little left to order them by. An attention mechanism over the question, trained to score each formula by how much of the question it carries, would pick the one or two that matter and query with those.

3. **One representation for a formula and its prose.** Every configuration in this thesis keeps the two apart. Text rankings and formula rankings are produced separately and then merged, and Section 7.1 gives the reason that arrangement can only go so far: a formula match locates the right neighbourhood, and the words around the formula decide which answer in it resolves the question. A retriever trained to embed a post as one object, with its formulas and its prose in a single representation, would not need the merge at all.

Chapter 8

Acknowledgement of AI Tool Usage

Declaration on the Use of Generative AI

Generative AI tools were used in this thesis only as a supporting resource, for the following purposes:

- Language revision, grammar correction, translation of the abstract into German, and rephrasing of sentences originally written by the author.
- Clarifying technical concepts related to the topic of this thesis, in the same way one would consult a textbook or a study companion.
- Debugging of code written by the author.
- Searching efficiently for related work in the field.

The following tools were used:

- Grammarly, for grammar, spelling and readability.
- Claude, for debugging code, paraphrasing and translation.
- Consensus, for finding published work in the field.

Bibliography

[1] Zahra Abbasiantaeb et al. “Can We Use Large Language Models to Fill Relevance Judgment Holes?” In: *arXiv preprint arXiv:2405.05600* (2024).

[2] Akiko Aizawa and Michael Kohlhase. “Mathematical Information Retrieval”. In: *Evaluating Information Retrieval and Access Tasks. NTCIR's Legacy of Research Impact*. Ed. by Tetsuya Sakai, Douglas W. Oard, and Noriko Kando. Vol. 43. The Information Retrieval Series. Singapore: Springer, 2021, pp. 169–185. DOI: 10.1007/978-981-15-5554-1_12.

[3] Shaden Alshammari et al. “MathNet: A Global Multimodal Benchmark for Mathematical Reasoning and Retrieval”. In: *International Conference on Learning Representations*. 2026, pp. 12477–12507.

[4] Piotr Bojanowski et al. “Enriching Word Vectors with Subword Information”. In: *Transactions of the Association for Computational Linguistics* 5 (2017), pp. 135–146. DOI: 10.1162/tacl_a_00051.

[5] Chris Buckley and Ellen M. Voorhees. “Evaluating Evaluation Measure Stability”. In: *Proceedings of the 23rd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2000, pp. 33–40. DOI: 10.1145/345508.345543.

[6] Chris Buckley and Ellen M. Voorhees. “Retrieval evaluation with incomplete information”. In: *Proceedings of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2004, pp. 25–32. DOI: 10.1145/1008992.1009000.

[7] Chris Burges et al. “Learning to Rank Using Gradient Descent”. In: *Proceedings of the 22nd International Conference on Machine Learning*. Bonn, Germany: ACM, 2005, pp. 89–96. DOI: 10.1145/1102351.1102363.

[8] Charles L. A. Clarke and Laura Dietz. “LLM-Based Relevance Assessment Still Can’t Replace Human Relevance Assessment”. In: *arXiv preprint arXiv:2412.17156* (2024). Version 3, 19 January 2026.

[9] Jacob Cohen. “A Coefficient of Agreement for Nominal Scales”. In: *Educational and Psychological Measurement* 20.1 (1960), pp. 37–46. DOI: 10.1177/001316446002000104.

[10] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2009, pp. 758–759. DOI: 10.1145/1571941.1572114.

[11] Pankaj Dadure, Partha Pakray, and Sivaji Bandyopadhyay. “Mathematical Information Retrieval: A Review”. In: *ACM Computing Surveys* 57.3, 61 (2024). DOI: 10.1145/3699953.

[12] Kenny Davila and Richard Zanibbi. “Layout and Semantics: Combining Representations for Mathematical Formula Search”. In: *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2017, pp. 1165–1168. DOI: 10.1145/3077136.3080748.

[13] Shahul Es et al. “RAGAs: Automated Evaluation of Retrieval Augmented Generation”. In: *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. St. Julians, Malta: Association for Computational Linguistics, 2024, pp. 150–158. DOI: 10.18653/v1/2024.eacl-demo.16.

[14] Exploding Gradients. *RAGAS: Evaluation Framework for RAG and LLM Applications*. Version 0.4.3. 2026. URL: https://github.com/explodinggradients/ragas.

[15] Alvan R. Feinstein and Domenic V. Cicchetti. "High Agreement but Low Kappa: I. The Problems of Two Paradoxes". In: *Journal of Clinical Epidemiology* 43.6 (1990), pp. 543–549. DOI: 10.1016/0895-4356(90)90158-L.

[16] Maik Fröbe et al. "Bootstrapped nDCG Estimation in the Presence of Unjudged Documents". In: *Advances in Information Retrieval. 45th European Conference on Information Retrieval, ECIR 2023*. Ed. by Jaap Kamps et al. Vol. 13980. Lecture Notes in Computer Science. Cham: Springer, 2023, pp. 313–329. DOI: 10.1007/978-3-031-28244-7_20.

[17] Maik Fröbe et al. "Large Language Model Relevance Assessors Agree With One Another More Than With Human Assessors". In: *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2025, pp. 2858–2863. DOI: 10.1145/3726302.3730218.

[18] Yunfan Gao et al. "Retrieval-Augmented Generation for Large Language Models: A Survey". In: *arXiv preprint arXiv:2312.10997* (2024). Version 5, 27 March 2024.

[19] Owen Henkel et al. "Retrieval-Augmented Generation to Improve Math Question-Answering: Trade-offs Between Groundedness and Human Preference". In: *Proceedings of the 17th International Conference on Educational Data Mining (EDM)*. Atlanta, Georgia, USA: International Educational Data Mining Society, 2024, pp. 315–320. DOI: 10.5281/zenodo.12729824.

[20] Kalervo Järvelin and Jaana Kekäläinen. "Cumulated Gain-Based Evaluation of IR Techniques". In: *ACM Transactions on Information Systems* 20.4 (2002), pp. 422–446. DOI: 10.1145/582415.582418.

[21] Jeff Johnson, Matthijs Douze, and Hervé Jégou. "Billion-Scale Similarity Search with GPUs". In: *IEEE Transactions on Big Data* 7.3 (2021), pp. 535–547. DOI: 10.1109/TBDATA.2019.2921572.

[22] Haocheng Ju and Bin Dong. "MIRB: Mathematical Information Retrieval Benchmark". In: *arXiv preprint arXiv:2505.15585* (2025).

[23] Vladimir Karpukhin et al. "Dense Passage Retrieval for Open-Domain Question Answering". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.

[24] Maurice G. Kendall. "A New Measure of Rank Correlation". In: *Biometrika* 30.1/2 (1938), pp. 81–93. DOI: 10.1093/biomet/30.1-2.81.

[25] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. 2020.

[26] Ruyin Li and Xiaoyu Chen. "SSEmb: A Joint Structural and Semantic Embedding Framework for Mathematical Formula Retrieval". In: *arXiv preprint arXiv:2508.04162* (2025).

[27] Jimmy Lin et al. "Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations". In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2021, pp. 2356–2362. DOI: 10.1145/3404835.3463238.

[28] Behrooz Mansouri et al. "DPRL Systems in the CLEF 2022 ARQMath Lab: Introducing MathAMR for Math-Aware Search". In: *Working Notes of CLEF 2022*. Vol. 3180. CEUR Workshop Proceedings. 2022.

[29] Behrooz Mansouri et al. "Effects of context, complexity, and clustering on evaluation for math formula retrieval". In: *arXiv preprint arXiv:2111.10504* (2021).

[30] Behrooz Mansouri et al. “Overview of ARQMath-2 (2021): Second CLEF Lab on Answer Retrieval for Questions on Math”. In: *Experimental IR Meets Multilinguality, Multimodality, and Interaction (CLEF 2021)*. Vol. 12880. Lecture Notes in Computer Science. Springer, 2021, pp. 215–238. DOI: 10.1007/978-3-030-85251-1_17.

[31] Behrooz Mansouri et al. “Overview of ARQMath-3 (2022): Third CLEF Lab on Answer Retrieval for Questions on Math”. In: *Experimental IR Meets Multilinguality, Multimodality, and Interaction (CLEF 2022)*. Vol. 13390. Lecture Notes in Computer Science. Springer, 2022, pp. 286–310. DOI: 10.1007/978-3-031-13643-6_20.

[32] Behrooz Mansouri et al. “Tangent-CFT: An Embedding Model for Mathematical Formulas”. In: *Proceedings of the 2019 ACM SIGIR International Conference on Theory of Information Retrieval (ICTIR)*. ACM, 2019, pp. 11–18. DOI: 10.1145/3341981.3344235.

[33] Bruce R. Miller. *LaTeXML: A LaTeX to XML/HTML/MathML Converter*. Version 0.8.8. 2024. URL: https://math.nist.gov/~BMiller/LaTeXML/.

[34] Shuai Peng et al. “MathBERT: A Pre-Trained Model for Mathematical Formula Understanding”. In: *arXiv preprint arXiv:2105.00377* (2021).

[35] Radim Řehůřek and Petr Sojka. “Software Framework for Topic Modelling with Large Corpora”. In: *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*. Valletta, Malta: ELRA, 2010, pp. 45–50.

[36] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410.

[37] Stephen Robertson and Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389. DOI: 10.1561/1500000019.

[38] Tetsuya Sakai. "Alternatives to Bpref". In: *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2007, pp. 71–78. DOI: 10.1145/1277741.1277756.

[39] Tetsuya Sakai and Noriko Kando. "On information retrieval metrics designed for evaluation with incomplete relevance assessments". In: *Information Retrieval* 11.5 (2008), pp. 447–470. DOI: 10.1007/s10791-008-9059-7.

[40] Alireza Salemi and Hamed Zamani. "Evaluating Retrieval Quality in Retrieval-Augmented Generation". In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. Washington, DC, USA: ACM, 2024. DOI: 10.1145/3626772.3657957.

[41] Julius Sim and Chris C. Wright. "The Kappa Statistic in Reliability Studies: Use, Interpretation, and Sample Size Requirements". In: *Physical Therapy* 85.3 (2005), pp. 257–268. DOI: 10.1093/ptj/85.3.257.

[42] Kaitao Song et al. "MPNet: Masked and Permuted Pre-training for Language Understanding". In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 2020, pp. 16857–16867.

[43] Annalisa Szymanski et al. "Limitations of the LLM-as-a-Judge Approach for Evaluating LLM Outputs in Expert Knowledge Tasks". In: *Proceedings of the 30th International Conference on Intelligent User Interfaces*. ACM, 2025, pp. 952–966. DOI: 10.1145/3708359.3712091.

[44] Paul Thomas et al. "Large Language Models Can Accurately Predict Searcher Preferences". In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2024, pp. 1930–1940. DOI: 10.1145/3626772.3657707.

[45] Shivani Upadhyay et al. "A Large-Scale Study of Relevance Assessments with Large Language Models Using UMBRELA". In: *Proceedings of the 2025 ACM SIGIR International Conference on Theory of Information Retrieval (ICTIR)*. ACM, 2025. DOI: 10.1145/3731120.3744605.

[46] Christophe Van Gysel and Maarten de Rijke. "Pytrec_eval: An Extremely Fast Python Interface to trec_eval". In: *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 2018, pp. 873–876. DOI: 10.1145/3209978.3210065.

[47] Pei-Syuan Wang and Hung-Hsuan Chen. "The Effectiveness of Graph Contrastive Learning on Mathematical Information Retrieval". In: *arXiv preprint arXiv:2402.13444* (2024).

[48] Richard Zanibbi, Behrooz Mansouri, and Anurag Agarwal. "Mathematical Information Retrieval: Search and Question Answering". In: *Foundations and Trends in Information Retrieval* 19.1-2 (2025), pp. 1–190. DOI: 10.1561/1500000095.

[49] Richard Zanibbi et al. "NTCIR-12 MathIR Task Overview". In: *Proceedings of the 12th NTCIR Conference on Evaluation of Information Access Technologies*. Tokyo, Japan, 2016, pp. 299–308.

[50] Richard Zanibbi et al. "Overview of ARQMath 2020: CLEF Lab on Answer Retrieval for Questions on Math". In: *Experimental IR Meets Multilinguality, Multimodality, and Interaction (CLEF 2020)*. Vol. 12260. Lecture Notes in Computer Science. Springer, 2020, pp. 169–193. DOI: 10.1007/978-3-030-58219-7_15.